Memory Management



Dr. Hiteshwar Kumar Azad
Email ID: hiteshwarkumar.azad@vit.ac.in
SCOPE

Outline

- Background
- Contiguous Memory Allocation
- Paging
- Structure of the Page Table
- Swapping

Objectives

- To provide a detailed description of various ways of organizing memory hardware
- To discuss various memory-management techniques
- To provide a detailed description of the Intel Pentium, which supports both pure segmentation and segmentation with paging

Memory Management – Background

Program Storage and Loading

- Programs are **permanently stored on backing store** (e.g. hard disks or SSDs).
- To **run a program**, it must be:
  - **Loaded from the backing store** into **main memory (RAM)**.
  - **Assigned to a process** by the operating system.

CPU Access to Memory

- The **CPU can only directly access**:
  - **Registers**
  - **Main memory (RAM)**
- **Registers** are the fastest: access time is **1 CPU clock cycle or less**.
- **Main memory access is slower**, often taking **multiple clock cycles**, which can cause **CPU stalls (waiting)**.

Background

How Memory Access Works

- The **memory unit** receives a stream of operations from the CPU:
  - **Read requests** → (Address + Read)
  - **Write requests** → (Address + Data to write)

Cache Memory

- **Cache memory** is a small, fast memory placed **between main memory and CPU registers**.
- Purpose: **Reduce average memory access time** by storing **frequently used instructions/data**.

Memory Protection

- **Memory protection mechanisms** are required to:
  - Prevent one process from accessing another process's memory.
  - Ensure **correct and stable operation** of the system.

Memory Protection

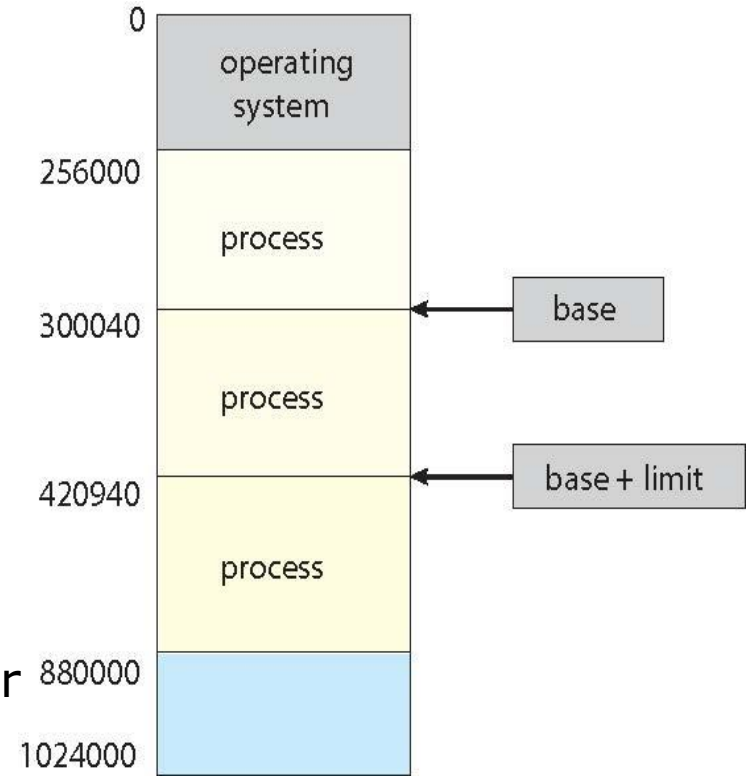
📌 **Purpose**

- To **ensure that a process can only access memory addresses within its own allocated address space**.
- Prevents **one process from accidentally or maliciously accessing another process's memory**.
- Helps maintain **system stability and security**.

⚙️ **How It Works — Base and Limit Registers**

•The CPU uses a **pair of registers** for each process:

- **Base register**
  - Holds the **starting physical address** of the process's memory space.
- **Limit register**
  - Specifies the **size (length) of the process's address space**.



A pair of **base** and **limit** registers work together to define a process's valid memory region.

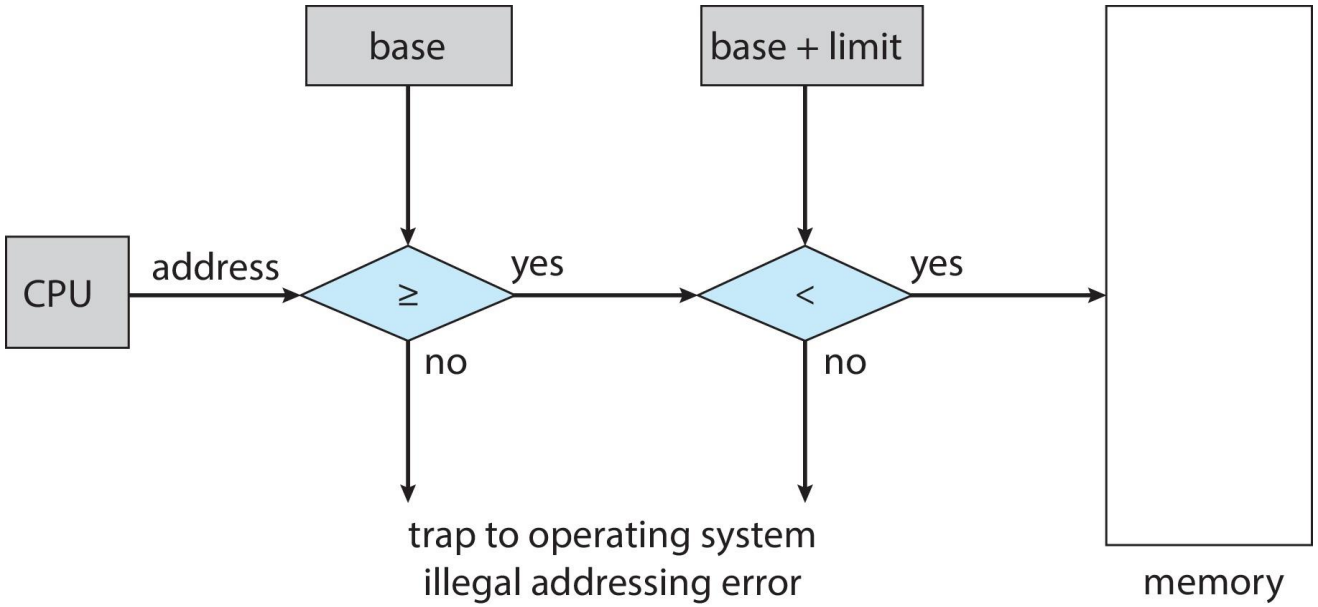
Hardware Address Protection

How it Works

- Each process has:
  - A **Base register** → stores the **starting physical address** of the process's memory.
  - A **Limit register** → stores the **size (range)** of the process's allocated memory.
- The **CPU checks every memory access** made in **user mode** as follows:

```
if (logical_address < limit)
        physical_address = base + logical_address
else
        trap → memory access violation
```

- This ensures that a **user process cannot access memory outside its assigned range.**



Address Binding

Concept

- **Address binding** is the process of **mapping program addresses from one form to another** during different stages of a program's life.
- Allows programs to be **loaded at different memory locations** instead of always at physical address 0000.

Why Needed

- Programs are stored on **disk (backing store)** and placed in an **input queue** before execution.
- Without binding support, programs would **always have to load at address 0000**, which is:
  - Inflexible
  - Allows only one program at a time

Types of Addresses:

Stage	Address Type	Example	Description
Source code	Symbolic	varA, loopStart	Human-readable names
Compiled code	Relocatable	14 bytes from start	Relative to start of the module
Loaded in memory	Absolute (Physical)	74014	Actual address in

Binding of Instructions and Data to Memory

- **Address binding** can occur at **three different stages** during a program's life cycle:

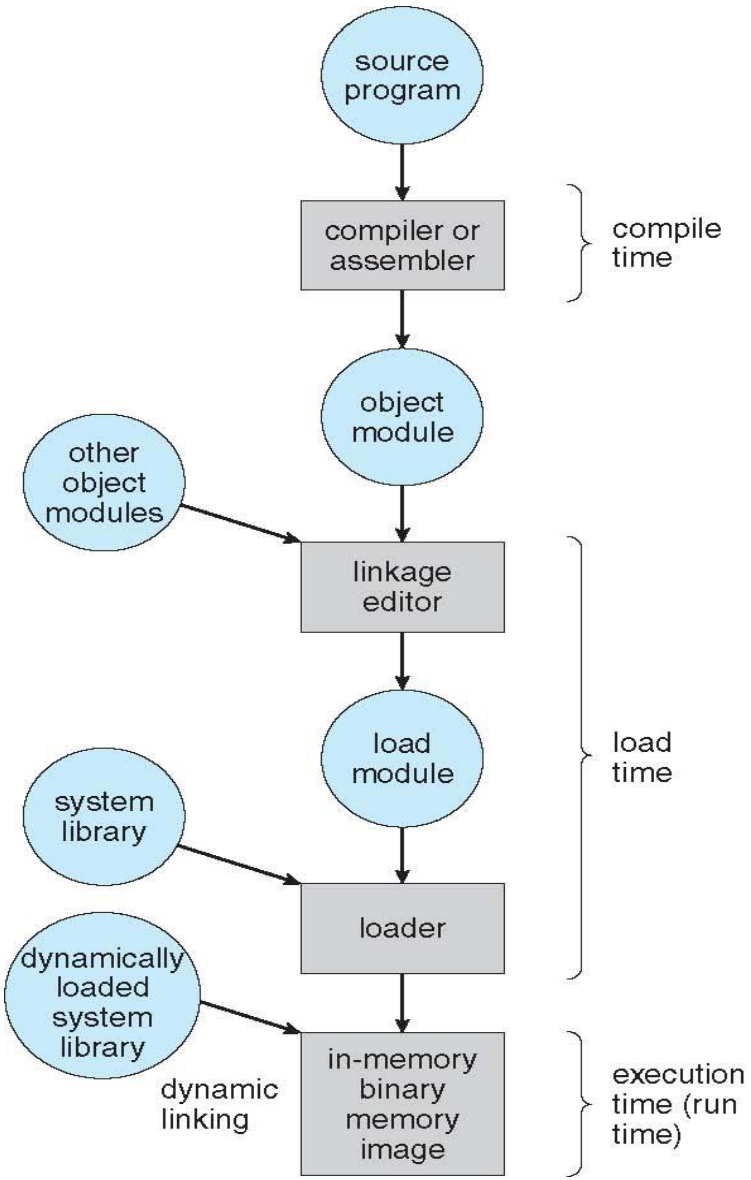
🧩 **Types of Binding**

- **Compile-time Binding**
  - Used when memory location is **known in advance**
  - Compiler generates **Absolute code**
  - If starting location changes → **must recompile the code**
- **Load-time Binding**
  - Used when memory location is **not known at compile time**
  - Compiler generates **Relocatable code**
  - **Loader** assigns absolute addresses when program is loaded into memory
- **Execution-time Binding**
  - Used when a process **can be moved during execution**
  - Binding is done **at run time** by the **Memory Management Unit (MMU)**
  - Requires **hardware support** (e.g. **Base register** and **Limit register**)

✅ **Key Point:**

- Each stage maps program addresses from **logical** → **physical**, allowing flexible loading and safe execution.

Multistep Processing of a User Program



Logical vs. Physical Address Space

Key Concepts

- **Logical Address (Virtual Address)**
  - Generated by the **Central processing unit (CPU)** during program execution
  - Used by programs to reference memory locations
- **Physical Address**
  - Actual address in main memory
  - Seen by the **Memory management unit (MMU)** and memory hardware

Address Binding

- **Compile-time & Load-time binding**
  - Logical and physical addresses are **the same**
- **Execution-time binding**
  - Logical (virtual) and physical addresses are **different**
  - Requires hardware support (e.g., base & limit registers, MMU)

Logical vs. Physical Address Space

Address Spaces

- **Logical Address Space**
  - Set of all logical (virtual) addresses generated by a program
- **Physical Address Space**
  - Set of all physical addresses mapped to actual memory cells

Flow:
CPU → Logical Address → [MMU translation] → Physical Address → Memory

Logical vs. Physical Address Space:

Aspect	Logical Address Space	Physical Address Space
Definition	Addresses generated by **(CPU)**	Actual addresses in **main memory (RAM)**
Also Called	Virtual addresses	Real addresses
Generated By	CPU	(MMU)
Visible To	Programs / user	Hardware / memory unit
When Known	Before translation	After translation

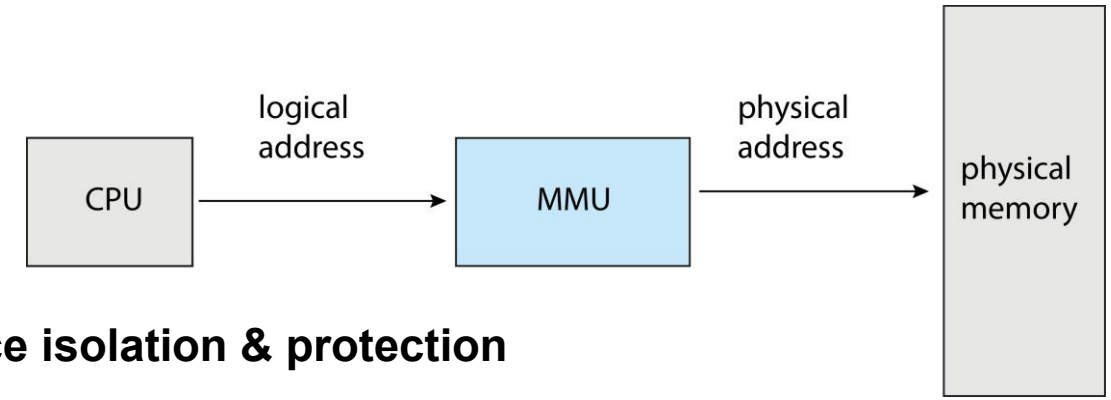
Memory-Management Unit (MMU)

Definition

- A **hardware component** that **translates logical (virtual) addresses** from the **CPU** into **physical addresses** in main memory at run time.

Working

- **CPU generates a logical address**
- **MMU translates** it to a physical address using mapping techniques
- **Physical memory** is accessed using the translated address
- 📌 This allows **programs to be relocated** anywhere in memory.



Notes

- Ensures **address space isolation & protection**
- **Multiple mapping methods** exist (e.g., base-limit registers, paging, segmentation)
- **Transparent to user** — the user sees only logical addresses

Flow

- **CPU → Logical Address → MMU → Physical Address → Main Memory**

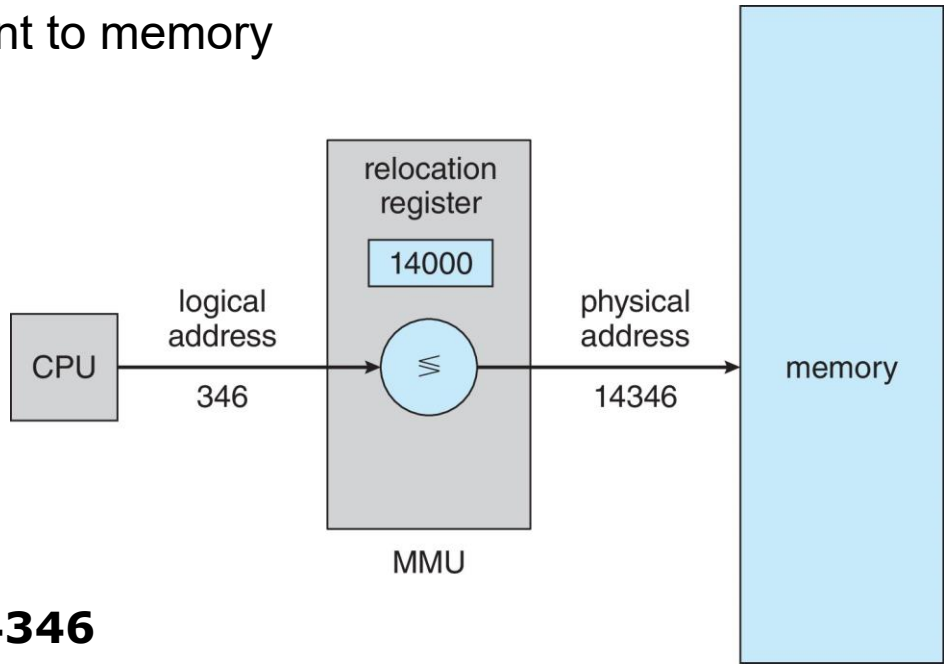
MMU — Relocation Register

Concept

- A simple address translation scheme
- **Generalization of the base-register scheme**
- The base register is called the **relocation register**

Working

- **CPU generates a logical address**
- **MMU** adds the value in the **relocation register** to it
- Resulting **physical address** is sent to memory



Example:

```
Relocation Register = 14000
Logical Address = 346
```

Physical Address = 14000 + 346 = 14346

Relocation Register (Cont.)

🧠 **Key Concepts**

- User programs **generate only logical addresses**
- They **never see real physical addresses**
- **Address binding happens at execution time**
  → when the CPU references a memory location

⚙️ **How It Works**

- On each memory access:
  - **Logical address + relocation register value = physical address**
- This **binds logical addresses to physical addresses dynamically** at runtime

✅ **Result:**

- Allows **relocation of programs in memory**
- Provides **memory protection**
- **Transparent to user** — translation is handled by MMU

Dynamic Loading

📍 Concept

- Program has a **main part** and multiple **routines (modules)**
- **Entire program need not be in memory** at once
- **Routine is loaded only when it is called**

💡 Advantages

- **Saves memory space** — unused routines are never loaded
- Allows **large programs** to run in **limited memory**
- **Efficient use of memory** for infrequently used code

⚙️ Implementation

- All routines stored on disk in **relocatable load format**
- **No special OS support needed** — handled by program design
- OS can **assist via libraries** that support dynamic loading

✅ **Summary**: Load **on-demand**, not **all at once** → **more efficient memory use**

⚡ Static vs Dynamic Linking

🧩 Static Linking

- **System libraries + program code** combined into **single binary** at **compile/load time**
- All code **copied into the executable**
- **Larger executable size**
- **No runtime dependency** on external libraries

🔗 Dynamic Linking

- **Linking deferred until execution time**
- Uses a small **stub** to locate memory-resident library routines
- **Stub loads routine if not already in memory**, then replaces itself with routine's address
- **Saves memory and disk space** (shared by many processes)
- Common for **shared libraries**
- **OS checks and loads library as needed**
- Allows **easy patching/updates** of libraries
- May require **versioning** to manage compatibility

⚡ Static vs Dynamic Linking

✅ Summary Table

Feature	Static Linking	Dynamic Linking
Linking Time	Compile/Load time	Execution time
Executable Size	Larger	Smaller
Library Updates	Requires rebuild	Automatic on update
Memory Use	Separate copy per process	Shared among processes
OS Support	Not required	Required

Memory Allocation

📌 Key Requirements

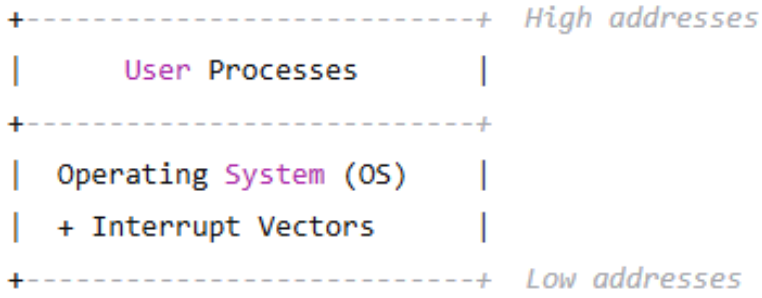
- **Main memory** is a **limited resource** shared between:
  - The **Operating System (OS)**
  - **User processes (programs)**
- Allocation must be done **efficiently** to maximize memory use and minimize waste.

🧩 Early Method: Contiguous Allocation

- Each process is loaded into **one single, continuous (contiguous) block of physical memory**.
- Simple to implement but can lead to **fragmentation** issues later.

🧩 Typical Memory Layout

- **Two main partitions** in main memory:
  - **Low memory**:
    - Holds the **resident Operating System**
    - Includes the **interrupt vector table** and system routines
  - **High memory**:
    - Holds **user processes** (programs)



Memory Allocation Techniques in OS

Operating systems manage memory allocation using different strategies:

A. Contiguous Memory Allocation

- Each process is stored in a single contiguous block of memory.
- **Advantages:**
  - Simple to implement
  - Fast access
- **Disadvantages:**
  - Memory fragmentation (both external and internal)
  - Not flexible for growing processes

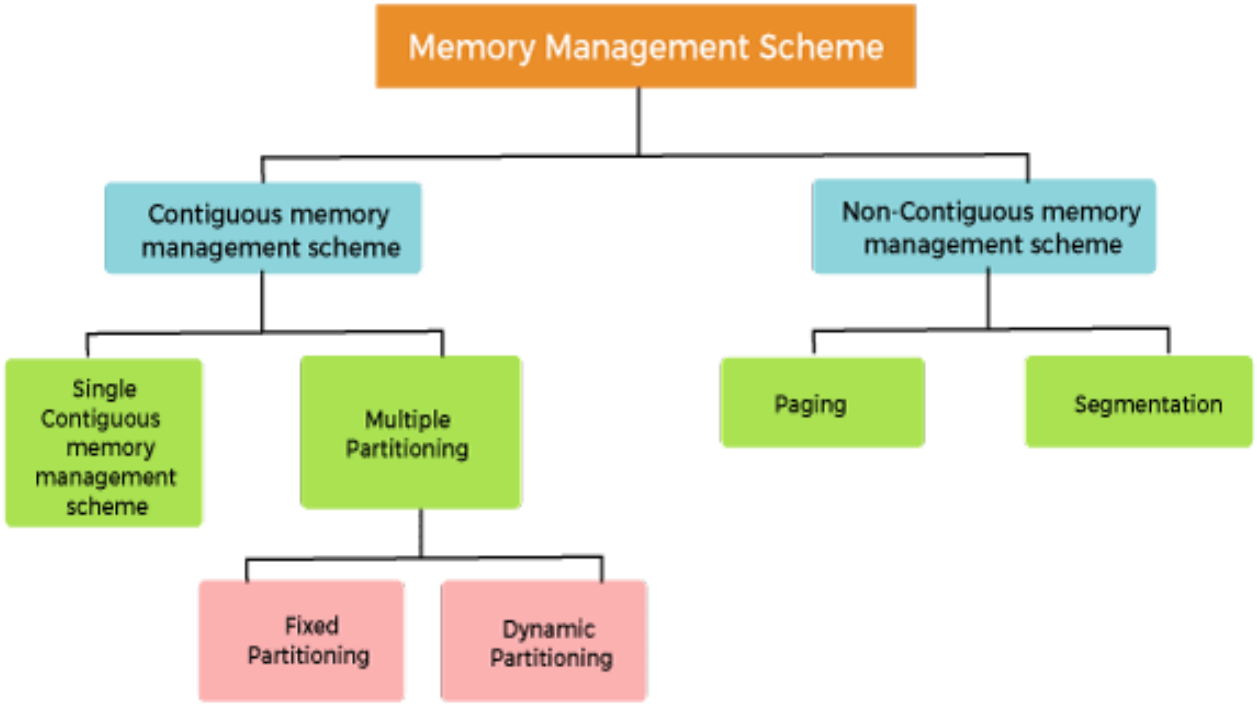
B. Non-Contiguous Memory Allocation

- Memory for a process can be in multiple non-contiguous blocks.
- Implemented using **paging** or **segmentation**.
- **Advantages:**
  - Avoids external fragmentation
  - Flexible for growing processes
- **Disadvantages:**
  - Slightly slower due to address translation
  - More complex to manage

Memory Allocation (cont.)

⚠ **Limitations of Contiguous Allocation**

- **External fragmentation**: free memory gets broken into small pieces over time.
- **Hard to grow processes dynamically**: each must fit into one chunk.
- **Limits multiprogramming**: fewer processes can be kept in memory at once.



Classification of memory management schemes

Memory Protection: Relocation & Limit Registers

- When multiple **user processes** and the **Operating System (OS)** share main memory, the system must ensure they **don't interfere** with each other's data or code.
- This is done using **hardware support**: **relocation** and **limit** registers, which are managed by the **Memory Management Unit (MMU)**.

Relocation Register (Base Register)

- Holds the **starting physical address** of a process in memory.
- Every logical address generated by the process is **added to this base address** to produce a **physical address**.
- Ensures the process **can only access memory inside its allocated region**.

Limit Register

- Holds the **size (length)** of the process's **logical address space**.
- Any logical address from the process must be **less than this value**.
- Prevents the process from generating addresses **outside its allowed range**.

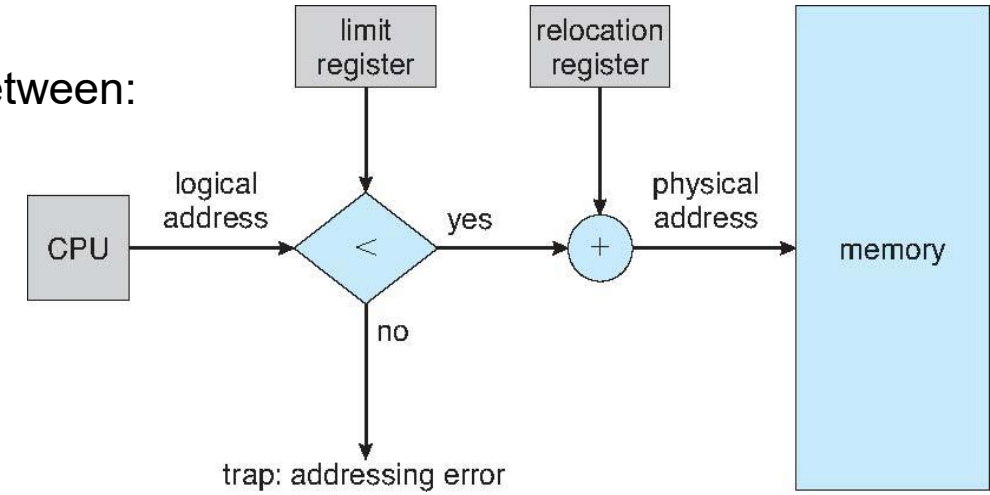
Memory Protection: Relocation & Limit Registers

⚡ Dynamic Mapping by the MMU

- The **MMU** combines these two registers to translate addresses **on the fly**:

```
if (logical_address < limit)
    physical_address = relocation + logical_address
else
    trap (memory violation)
```

- This allows the **Operating System** to **move processes around in memory** (relocate them) without changing their code.
- Provides **strong memory protection** between:
  - Different **user processes**
  - **User processes and the OS**

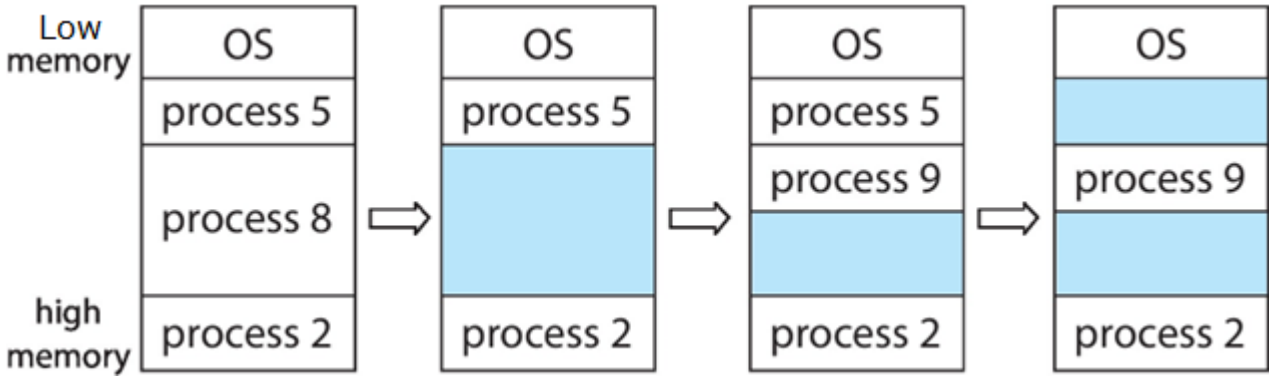


🧠 Why This Matters

- Prevents buggy or malicious programs from corrupting other processes or the OS.
- Enables **multiprogramming** safely.
- Forms the foundation for **modern memory protection mechanisms**.

Variable Partition Allocation: An Overview

- **Key Concepts:**
  - **Variable-size partitions:** Memory is divided into chunks that fit each process perfectly, improving efficiency.
  - **Multiprogramming:** The number of processes running at once is limited by the available memory partitions.
  - **Holes:** These are blocks of free, available memory scattered throughout the system.
- **Process Management:**
  - **Allocation:** A process is placed into a "hole" that's big enough to hold it.
  - **De-allocation:** When a process finishes, its memory partition is freed and merged with any adjacent free partitions to create larger holes.
  - **OS Responsibility:** The operating system tracks both occupied and free memory partitions.



Dynamic Storage-Allocation Problem

Goal: Allocate memory request of size *n* from available free holes

- 📌 **First-fit** → Allocate the first hole large enough
  - **Next-fit** → Similar **First Fit**, but search starts from the **last allocated block**
- 🎯 **Best-fit** → Allocate the smallest hole that fits; may require searching the entire list
  - Produces the **smallest leftover hole**
- 🛑 **Worst-fit** → Allocate the largest hole; may require full search
  - Produces the **largest leftover hole**

💡 **Insight:** First-fit and best-fit are generally faster and use memory more efficiently than worst-fit

First Fit Memory Allocation

- Allocates the **first hole** that is large enough for the process
- **Working principle**:
  - Traverse memory from the beginning
  - Allocate in the first free block that satisfies request
- **Simple and fast**

First Fit Algorithm (Steps):

- Start scanning memory from the **beginning**
- Check each free block (hole)
- If hole ≥ process size → allocate process
- If hole > process size → split block, leftover becomes a new hole
- If no hole found → process waits (or swapped)

First Fit (cont.)

Example

- **Memory Holes**: 100 KB, 500 KB, 200 KB, 300 KB, 600 KB
  Processes: P1=212 KB, P2=417 KB, P3=112 KB, P4=426 KB
- **Allocation using First Fit:**
  - P1 → 500 KB (left 288 KB)
  - P2 → 600 KB (left 183 KB)
  - P3 → 288 KB (from leftover of 500 KB block)
  - P4 → Not allocated (no hole large enough)
- **Advantages**
  - **Fast**: Scans only until a suitable hole is found
  - **Simple to implement**
  - Better memory utilization than Worst Fit
- **Disadvantages**
  - May cause **external fragmentation**
  - Tends to leave **small unusable holes** in memory
  - Performance degrades as memory fills → scanning takes longer

Example: First Fit

Problem:

- **Memory Holes (free blocks):**
  100 KB, 500 KB, 200 KB, 300 KB, 600 KB
- **Processes (to allocate):**
  212 KB, 417 KB, 112 KB, 426 KB

Step-by-Step Allocation:

1.Process 212 KB

1. Check holes in order:
    1. 100 KB → too small
    2. 500 KB → fits ✅
2. Allocate to 500 KB hole.
3. Remaining hole size = 500 - 212 = **288 KB**.
4. Updated holes: [100, 288, 200, 300, 600]

2.Process 417 KB

- Check holes in order:
    - 100 KB → too small
    - 288 KB → too small
    - 200 KB → too small
    - 300 KB → too small
    - 600 KB → fits ✅
- Allocate to 600 KB hole.
- Remaining hole size = 600 - 417 = **183 KB**.
- Updated holes: [100, 288, 200, 300, 183]

Example: First Fit (cont.)

3.Process 112 KB

•Check holes in order:

- 100 KB → too small
- 288 KB → fits ✅

•Allocate to 288 KB hole.

•Remaining hole size = 288 - 112 = **176 KB**.

•Updated holes: [100, 176, 200, 300, 183]

4.Process 426 KB

•Check holes in order:

- 100 KB → too small
- 176 KB → too small
- 200 KB → too small
- 300 KB → too small
- 183 KB → too small

•No suitable hole → **cannot be allocated** ❌

Final Allocation Result:

- **Process 212 KB** → allocated in 500 KB hole (left 288 KB → later 176 KB after next allocation).
- **Process 417 KB** → allocated in 600 KB hole (left 183 KB).
- **Process 112 KB** → allocated in 288 KB hole (left 176 KB).
- **Process 426 KB** → **not allocated**.

Next Fit Algorithm

- Similar to **First Fit**, but search starts from the **last allocated position**
- Wraps around to the beginning if needed
- Prevents frequent use of lower memory addresses

- **Steps**
  - Start search from **last allocated hole**
  - Find first hole ≥ process size
  - Allocate (split if needed)
  - Continue future searches from this point
- **Example**
  - **Holes**: 100, 500, 200, 300, 600 KB
    Processes: 212, 417, 112, 426 KB

Final Allocation :

- **Process 212 KB** → allocated in 500 KB hole (left 288 KB → later 176 KB).
- **Process 417 KB** → allocated in 600 KB hole (left 183 KB).
- **Process 112 KB** → allocated in 288 KB hole (left 176 KB).
- **Process 426 KB** → **not allocated**.

Example: Next Fit

Problem:

- **Memory Holes (free blocks):**
  100 KB, 500 KB, 200 KB, 300 KB, 600 KB
- **Processes (to allocate):**
  212 KB, 417 KB, 112 KB, 426 KB

Step-by-Step Allocation:

1.Process 212 KB

- Start from first hole (no previous allocation yet).
- 100 KB → too small
- 500 KB → fits ✅
- Allocate here.
- Remaining hole = 500 - 212 = **288 KB**.
- Updated holes: [100, 288, 200, 300, 600]
- **Next search will start from 288 KB hole.**

2.Process 417 KB

- Start from hole after 288 KB.
- 200 KB → too small
- 300 KB → too small
- 600 KB → fits ✅
- Allocate here.
- Remaining hole = 600 - 417 = **183 KB**.
- Updated holes: [100, 288, 200, 300, 183]
- **Next search will start from 183 KB hole.**

Example: Next Fit (cont.)

3.Process 112 KB

- Start from hole after 183 KB (wrap around if needed).
- No holes after 183 → wrap to start.
- 100 KB → too small
- 288 KB → fits ✅
- Allocate here.
- Remaining hole = 288 - 112 = **176 KB**.
- Updated holes: [100, 176, 200, 300, 183]
- **Next search will start from 176 KB hole.**

4.Process 426 KB

- Start from hole after 176 KB.
- 200 KB → too small
- 300 KB → too small
- 183 KB → too small
- Wrap around → 100 KB → too small
- 176 KB → too small
- ❌ No suitable hole found.

Final Allocation Result (Next Fit):

Process 212 KB → allocated in 500 KB hole (left 288 KB → later 176 KB).
Process 417 KB → allocated in 600 KB hole (left 183 KB).
Process 112 KB → allocated in 288 KB hole (left 176 KB).
Process 426 KB → **not allocated**.

Best Fit Algorithm

- Allocates the **smallest hole** that is big enough for the process
- Requires scanning the **entire memory list**
- Aims to reduce wasted space (internal fragmentation)

- **Steps**
  - Search all free holes
  - Find the **smallest hole ≥ process size**
  - Allocate (split if needed)
  - If no suitable hole → process waits
- **Example**
  - **Holes**: 100, 500, 200, 300, 600 KB
    Process (212 KB) → allocated in **300 KB** (best fit)
  - Leftover = 88 KB (new hole)

Pros & Cons

- **Advantages:**
  - Less internal fragmentation
  - Efficient use of memory in some cases

Disadvantages:

- Slower (must scan all holes)
- Creates many **small unusable holes** (external fragmentation)

Example: Best Fit

Problem:
•**Memory Holes (free blocks):**
100 KB, 500 KB, 200 KB, 300 KB, 600 KB
•**Processes (to allocate):**
212 KB, 417 KB, 112 KB, 426 KB

Step 1: Allocate 212 KB
Available holes: 100, 500, 200, 300, 600
•Holes ≥ 212 KB → 500, 300, 600
•**Smallest suitable hole** = 300 KB
Allocate 212 KB → 300 KB hole
•Remaining hole = 300 - 212 = **88 KB**
Updated holes: 100, 500, 200, 88, 600

Step 2: Allocate 417 KB
Available holes: 100, 500, 200, 88, 600
•Holes ≥ 417 KB → 500, 600
•**Smallest suitable hole** = 500 KB
Allocate 417 KB → 500 KB hole
•Remaining hole = 500 - 417 = **83 KB**
Updated holes: 100, 83, 200, 88, 600

Example: Best Fit (cont.)

Step 3: Allocate 112 KB

Available holes: 100, 83, 200, 88, 600

- Holes ≥ 112 KB → 200, 600
- **Smallest suitable hole** = 200 KB

Allocate 112 KB → 200 KB hole

- Remaining hole = 200 - 112 = **88 KB**

Updated holes: 100, 83, 88, 88, 600

Step 4: Allocate 426 KB

Available holes: 100, 83, 88, 88, 600

- Holes ≥ 426 KB → 600
- **Allocate 426 KB → 600 KB hole**
- Remaining hole = 600 - 426 = **174 KB**

Updated holes: 100, 83, 88, 88, 174

Final Allocation :

- Process **212 KB** → allocated in **300 KB hole** (left **88 KB**).
- Process **417 KB** → allocated in **500 KB hole** (left **83 KB**).
- Process **112 KB** → allocated in **200 KB hole** (left **88 KB**).
- Process **426 KB** → allocated in **600 KB hole** (left **174 KB**).

Worst Fit

- Allocates process to the **largest available hole**
- Idea: leave **biggest leftover** for future processes
- Opposite of Best Fit

- **Steps**
  - Search all free holes
  - Find the **largest hole ≥ process size**
  - Allocate (split if needed)
  - If no suitable hole → process waits
- **Example**
  - **Holes**: 100, 500, 200, 300, 600 KB
    Process (212 KB) → allocated in **600 KB** (worst fit)
  - Leftover = 388 KB (new hole)

Pros & Cons

- **Advantages:**
  - Leaves larger free spaces for later allocations
  - May reduce chances of large request rejection

Disadvantages:

•Poor memory utilization
•External fragmentation still occurs
•Search time is long (must check all holes)

Example

Problem:

- **Memory Holes (free blocks):**
  100 KB, 500 KB, 200 KB, 300 KB, 600 KB
- **Processes (to allocate):**
  212 KB, 417 KB, 112 KB, 426 KB

✅ **Final Allocation (Worst Fit):**

- Process **212 KB** → allocated in **600 KB hole** (left **388 KB**)
- Process **417 KB** → allocated in **500 KB hole** (left **83 KB**)
- Process **112 KB** → allocated in **388 KB hole** (left **276 KB**)
- Process **426 KB** → **not allocated**

Example

- Given memory partitions of 100K, 500K, 200K, 300K, and 600K (in order), how would each of the First-fit, Best-fit, and Worst-fit algorithms place processes of 212K, 417K, 112K, and 426K (in order)? Which algorithm makes the most efficient use of memory?

First-Fit:
212K is put in 500K partition.
417K is put in 600K partition.
112K is put in 288K partition (new partition 288K = 500K - 212K).
426K must wait.

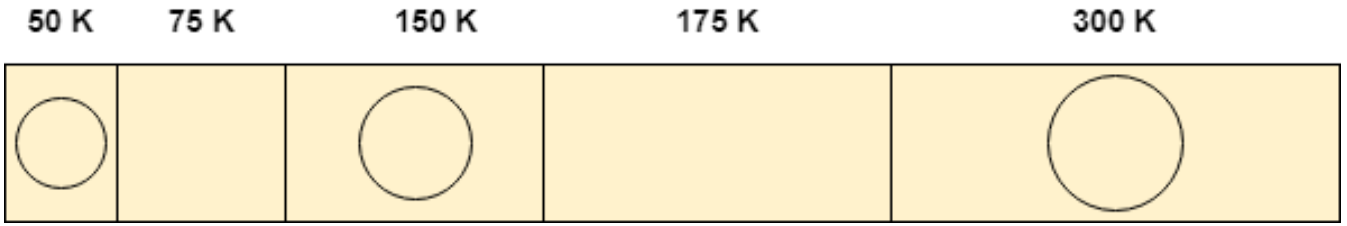
Best-Fit:
212K is put in 300K partition.
417K is put in 500K partition.
112K is put in 200K partition.
426K is put in 600K partition.

Worst-Fit:
212K is put in 600K partition.
417K is put in 500K partition.
112K is put in 388K partition.
426K must wait.

In this example, Best-Fit turns out to be the best.

Example

- Q1. Process requests are given as; **25 K , 50 K , 100 K , 75 K**



Determine the algorithm which can optimally satisfy this requirement.

1. First Fit algorithm
2. Best Fit Algorithm
3. Neither of the two
4. Both of them

Memory Allocation Problem

- A system has **five memory partitions**: 120 KB, 500 KB, 220 KB, 320 KB, 400 KB.

  Four processes arrive in order with memory requirements:
  A = 212 KB, B = 95 KB, C = 430 KB, D = 120 KB.

  Allocate memory using the following strategies:

  a) **First-Fit**
  b) **Best-Fit**
  c) **Worst-Fit**

  For each strategy, show the **partition allocated to each process**, the **remaining free space**, and determine **which strategy uses memory most efficiently**.



Which of the following is a disadvantage of the Best Fit algorithm?

i) Leads to large unused memory blocks
ii) Time-consuming search for the smallest suitable hole
iii) Always results in highest fragmentation
iv) It does not allocate memory at all

A) i

C) iv

B) iii

D) ii

Next

Suppose memory has holes of sizes 100 KB, 500 KB, 200 KB, 300 KB, and 600 KB (in order). If a process requests 212 KB, which hole will be allocated using Best Fit?

A) 500 KB

C) 200 KB

B) 600 KB

D) 300 KB

Next

Memory holes (free blocks) of sizes **6 KB, 15 KB, 12 KB** are available. Three requests arrive in this order: **10 KB, 2 KB, 14 KB**. Which allocation strategy among Best Fit, Worst Fit, First Fit, Next Fit would be able to satisfy *all three* requests?

A) Best Fit

C) Worst Fit

B) First Fit

D) Next Fit

Next



Suppose you have memory holes of sizes: **120 KB**, **25 KB**, **90 KB**, **30 KB**, **45 KB**, **75 KB** (in that order). Three processes arrive requiring **35 KB**, **15 KB**, **80 KB**, in that order. Using **Worst Fit** allocation, which hole is allocated to the third process (80 KB)?

A) 90KB

C) 75KB

B) 45KB

D) 120KB

Next

There are fixed partitions of sizes **100 KB, 260 KB, 60 KB, 180 KB, 30 KB**. Processes P1, P2, P3, P4 arrive in order, with sizes **180 KB, 20 KB, 120 KB, 130 KB**. Using **First Fit**, which process among them will *not* get memory allocated?

A) P4

C) P2

B) P1

D) P3

Next



Consider fixed partitions: **300, 400, 450, 500, 200, 250 KB**. Processes of sizes **200, 420, 220, 230 KB** arrive in that order. Using **Next Fit**, which partitions are *not* allocated to any process?

A) 200 & 400 KB

C) 200 & 300 KB

B) 200 & 250 KB

D) 200,250 & 400 KB

Next

Consider 6 memory partitions of size 200, 400, 600, 500, 300 and 250 KB. These partitions need to be allotted to four processes of sizes 357, 210, 468, 491 KB in that order. If the best-fit algorithm is used, which partitions are NOT allotted to any process?

A) 200 & 300 KB

C) 200 & 250 KB

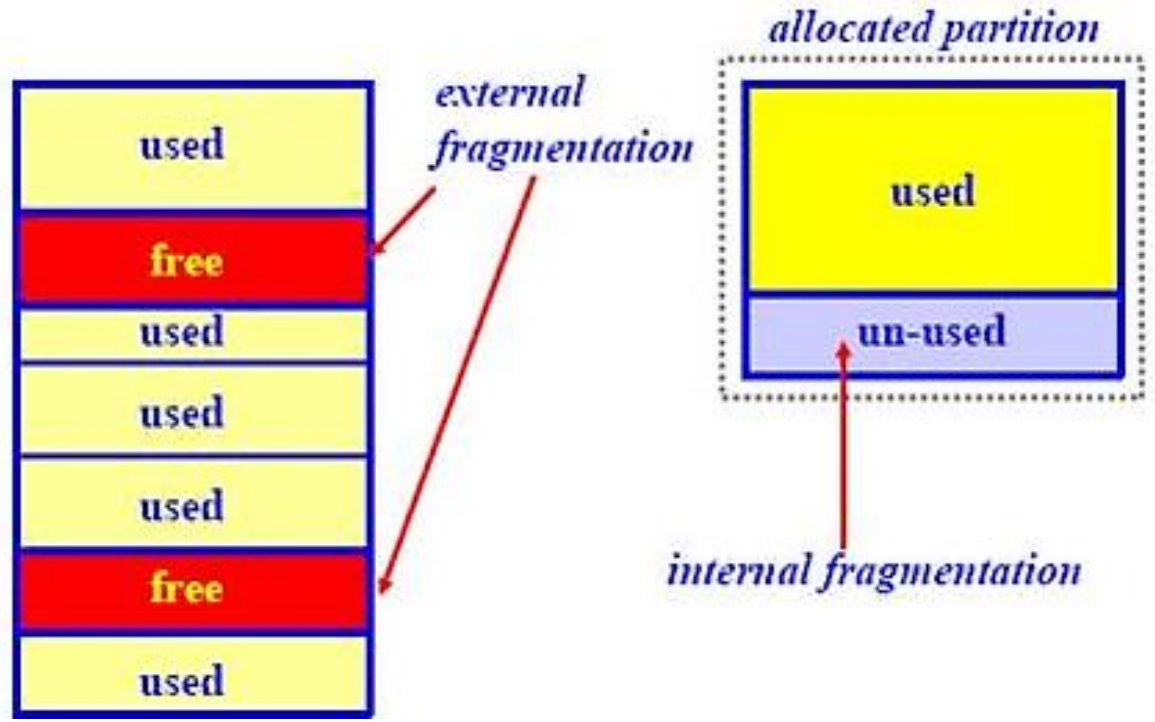
B) 250 & 300 KB

D) 300 & 400 KB

Next

Fragmentation in Memory Allocation

- **Definition:**
  Fragmentation occurs when memory is **inefficiently allocated**, leaving some parts of memory **unused or wasted**, even though the total memory may be sufficient.
- **Types of Fragmentation:**
  - External Fragmentation
  - Internal Fragmentation



Fragmentation

🟦 External Fragmentation

- Free memory exists but is **non-contiguous**.
- Total free memory may satisfy a request, but **no single block is large enough**.
- 💡 **Solution:** Compaction (rearranging memory to create contiguous free space).

🟩 Internal Fragmentation

- Allocated memory block is **slightly larger than requested**.
- Unused portion inside the block is **wasted memory**.

⚡ First-Fit Analysis & 50% Rule

- With **N allocated blocks**, roughly **0.5N blocks lost** to fragmentation.
- About **50% of memory may be unusable** over time.

✅ Key Takeaways

- External fragmentation → free memory **between blocks unusable**.
- Internal fragmentation → memory **wasted inside blocks**.
- Compaction and **non-contiguous allocation** (paging/segmentation) help reduce fragmentation.

Compaction

- 🔄 Rearranges memory to consolidate free space into **one large block**
- ⚡ Requires **dynamic relocation** at execution time

⚠️ **I/O Problem**

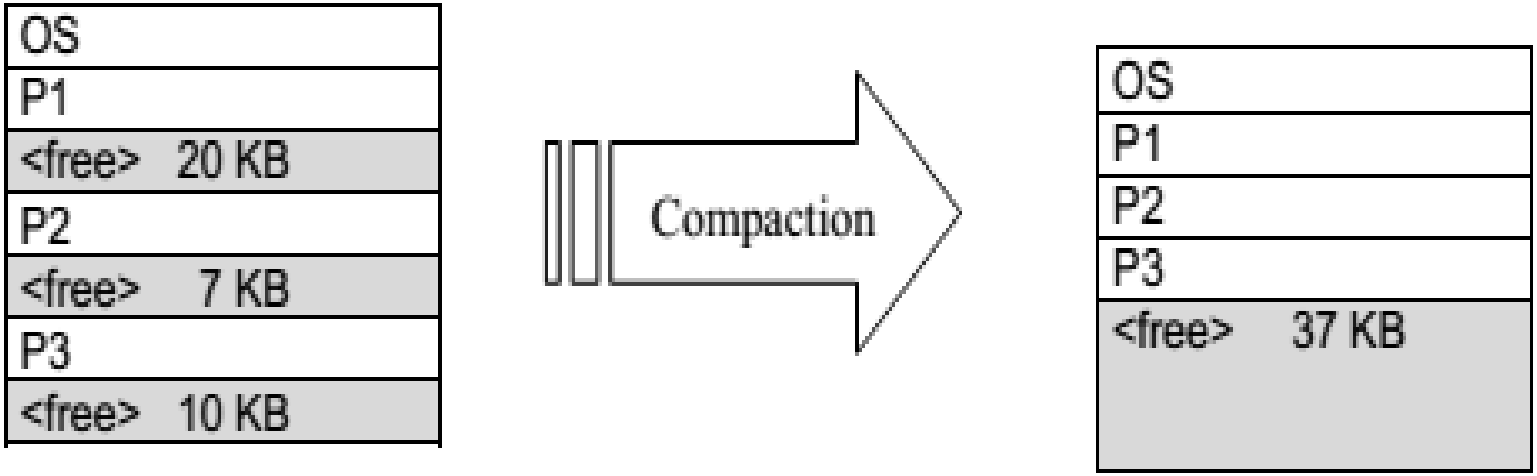
- If compaction happens during I/O → **data may be written to wrong location**

🛠 **Solutions**

- 🔒 **Latch process** in memory while it's doing I/O
- 📥 Do I/O only into **OS buffers** (safe staging area)

💾 **Note**

- Backing store (disks) also suffer from **fragmentation issues**



Non-contiguous Allocation

- ✅ A process can be stored in **different memory locations** instead of one block

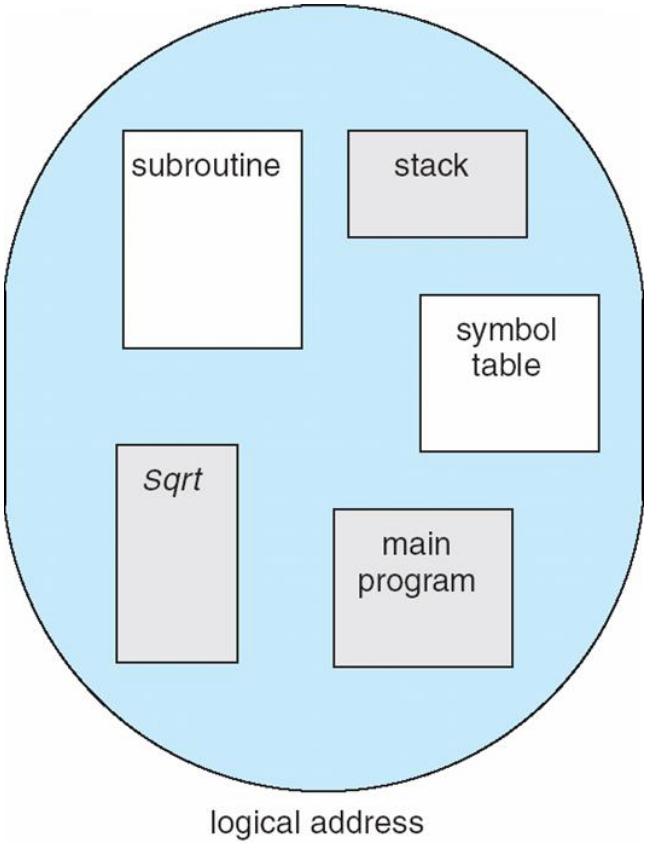
📌 **Techniques**

- 📐 **Segmentation** – divides process into **logical segments** (code, data, stack)
- 📄 **Paging** – splits memory into **fixed-size pages/frames**
- 🌀 **Paged Segmentation** – combines **segmentation** + **paging** for flexibility

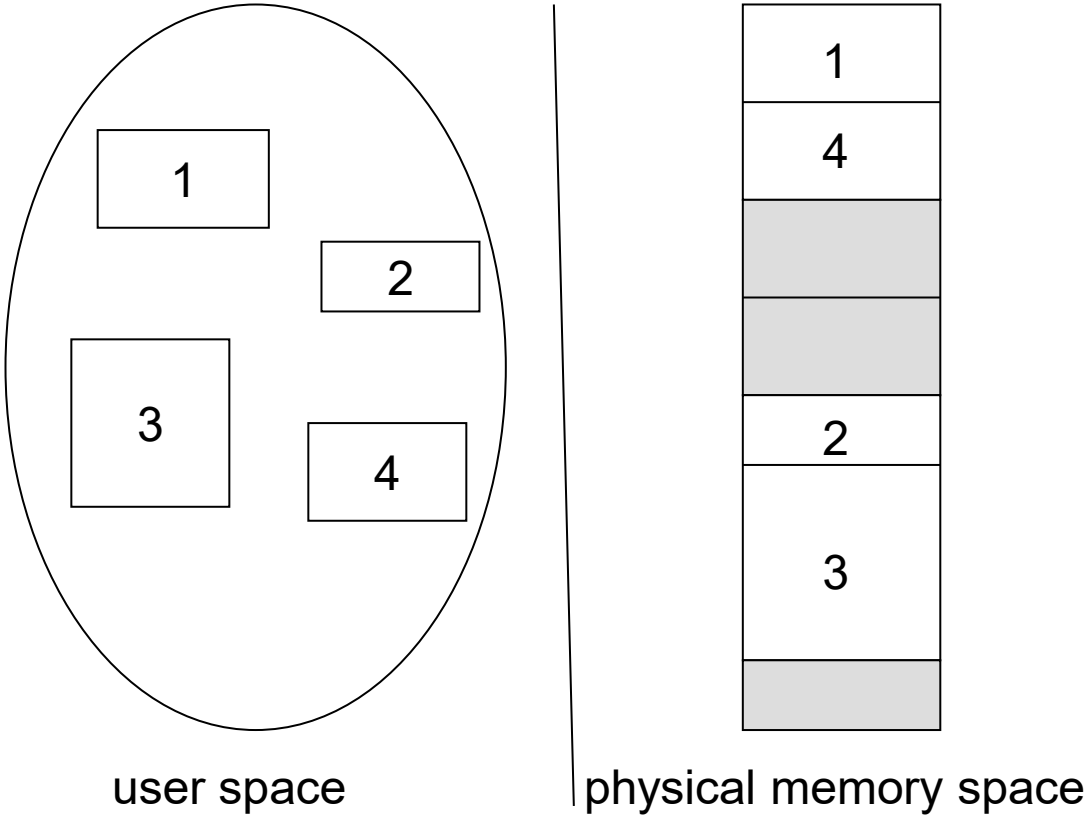
Segmentation

- **Definition:**
  A memory management scheme that divides a program into **logical segments**.
- **Key Idea:**
  - Each segment represents a **different logical unit** of a program
  - Segments vary in size
- **Examples of Segments:**
  - Main program / procedures / functions
  - Methods / objects
  - Local & global variables
  - Common blocks
  - Stack
  - Symbol tables
  - Arrays
- **Benefit:**
  Supports the **user's logical view of memory**, not just physical memory.

User's View of a Program



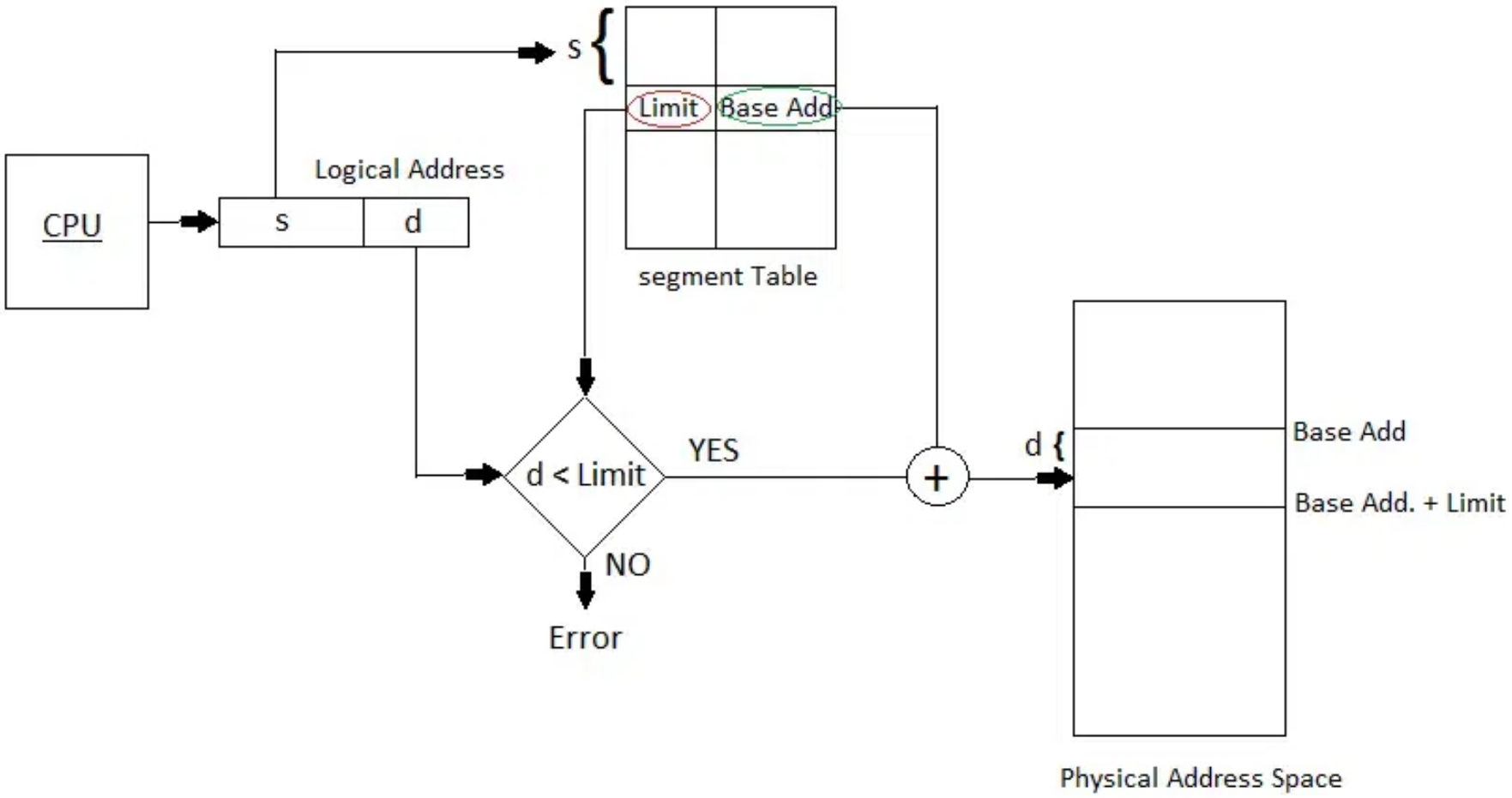
Logical View of Segmentation



Segmentation Architecture

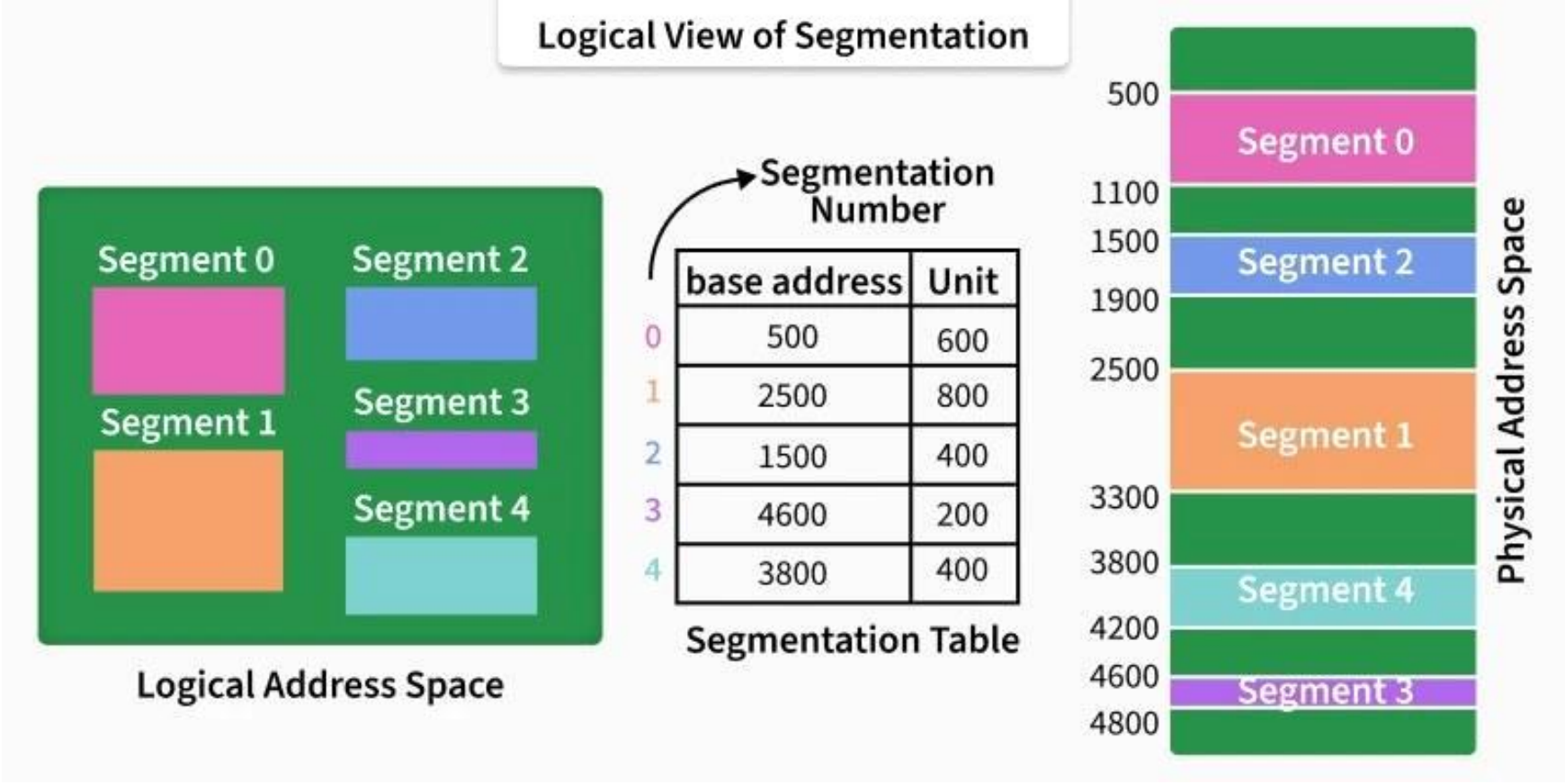
- **Logical Address**
  - Consist of two tuple: (Segment Number, Offset)
- **Segment Table**
  - Maps logical address → physical address
  - Each entry has:
    - **Base** → starting physical address of segment
    - **Limit** → length of the segment
- **Registers**
  - **STBR (Segment-Table Base Register)** → points to segment table in memory
  - **STLR (Segment-Table Length Register)** → total number of segments
    - Segment **s** is valid if s < STLR

Segmentation Hardware



Segmentation

- Divides program into **logical segments** (code, data, stack, etc.)
- Segments are stored in **non-contiguous physical memory**
- Each segment has a **variable size** (unlike paging)
- Can cause **external fragmentation**



Paging

- **Noncontiguous allocation**: A process's physical address space can be scattered across memory.
- **Advantages**
  - Eliminates **external fragmentation**
  - Allows **efficient memory utilization**
  - No issues with **variable-sized memory chunks**
- **Memory organization**
  - Physical memory → fixed-size **frames** (power of 2, between 512B–16MB)
  - Logical memory → same-sized **pages**
  - Backing store also divided into blocks
- **Execution**
  - To run a program of size **N pages**, find **N free frames** and load them.
  - Use a **page table** to map logical → physical addresses.
- **Address Translation**
  - CPU generates a **logical address (page number + offset)**
  - **Page Table** maps **page → frame**
  - Frame number + offset = **Physical Address**
- **Drawback**: Still suffers from **internal fragmentation** (last frame may not be fully used).

Paging

⚠ Drawback: Internal Fragmentation

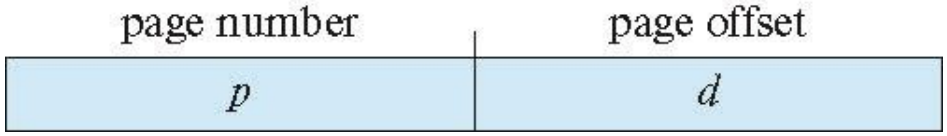
- Suppose **page/frame size = 1 KB**.
- Process size = **4.5 KB**.
- It needs **5 pages** (because pages can't be split).
- Memory allocated = **5 × 1 KB = 5 KB**.
- But only **4.5 KB is used** → **0.5 KB wasted** inside the last frame.

➡ **Wasted space = (1 – utilization) × allocated = 0.5 KB**

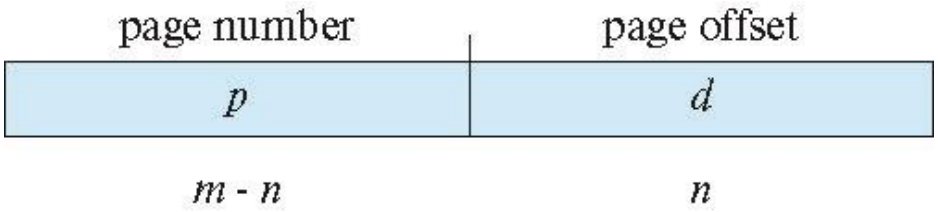
➡ This "unused portion" inside allocated frames is called **internal fragmentation**.

Address Translation Scheme

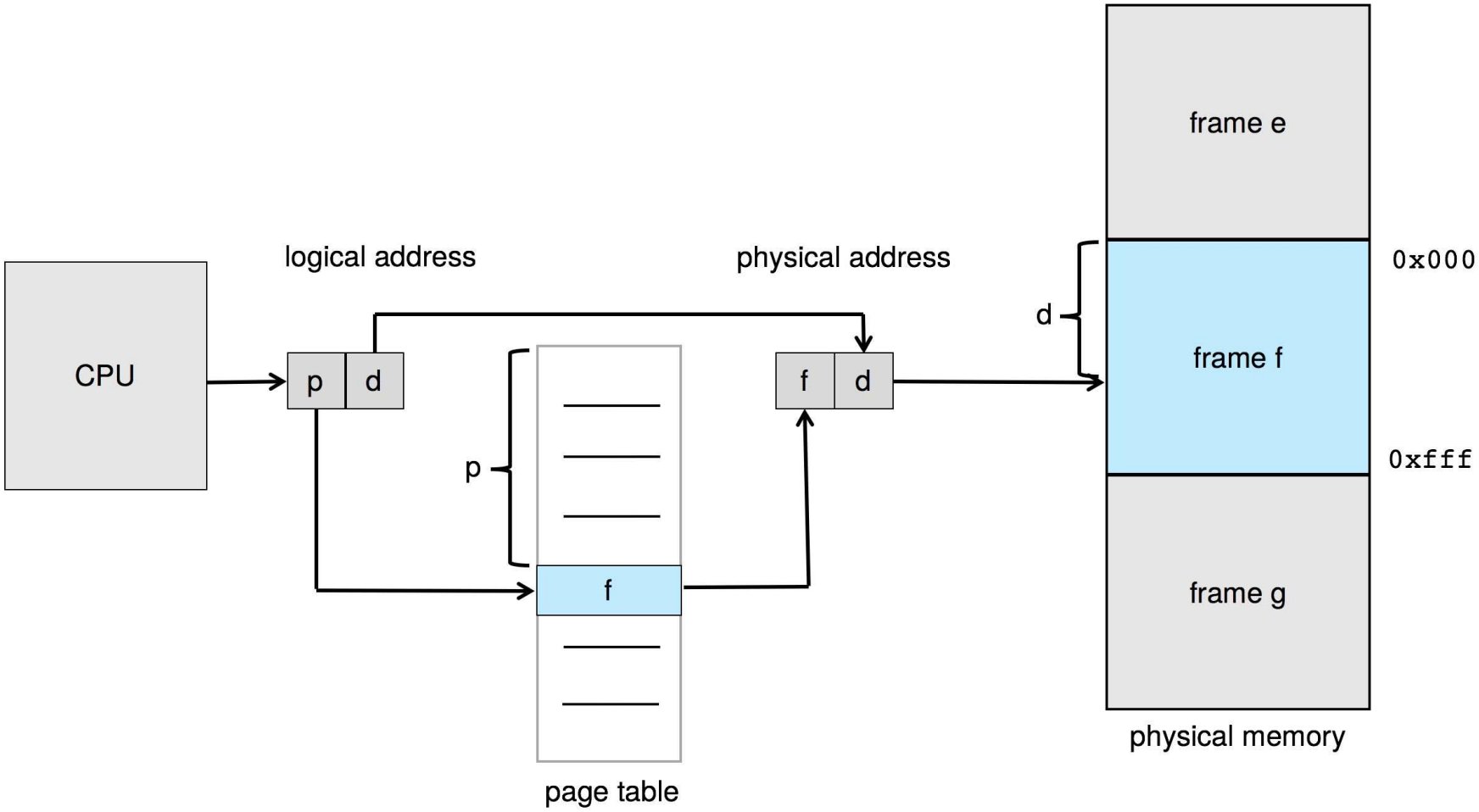
- CPU generates a **logical address (page number, offset)**
- Address generated by CPU is divided into:
  - **Page number** (***p***) – used as an index into a **page table** which contains base address of each page in physical memory
  - **Page offset** (***d***) – combined with base address to define the physical memory address that is sent to the memory unit



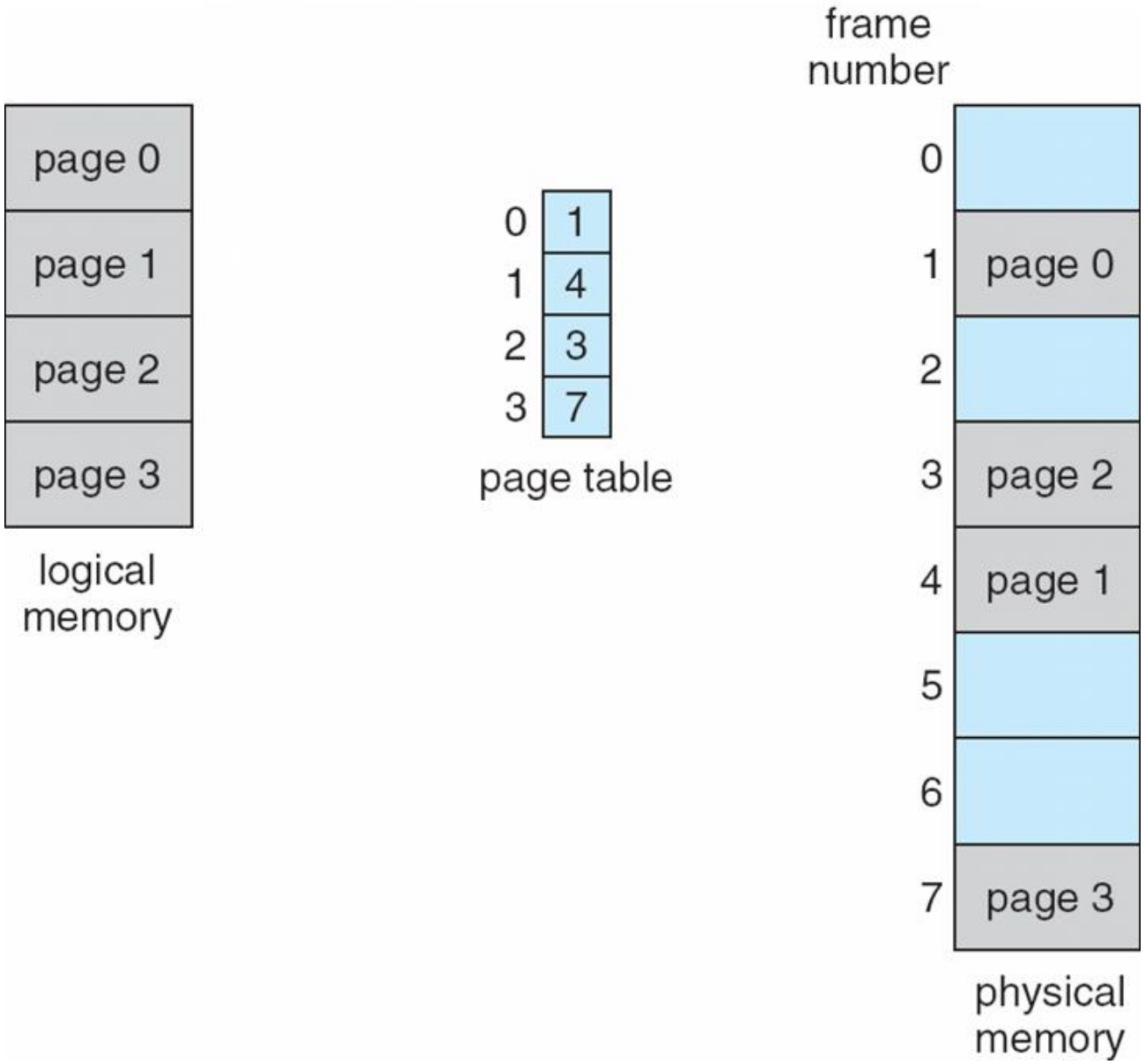
- For given logical address space 2^m and page size 2^n



Paging Hardware

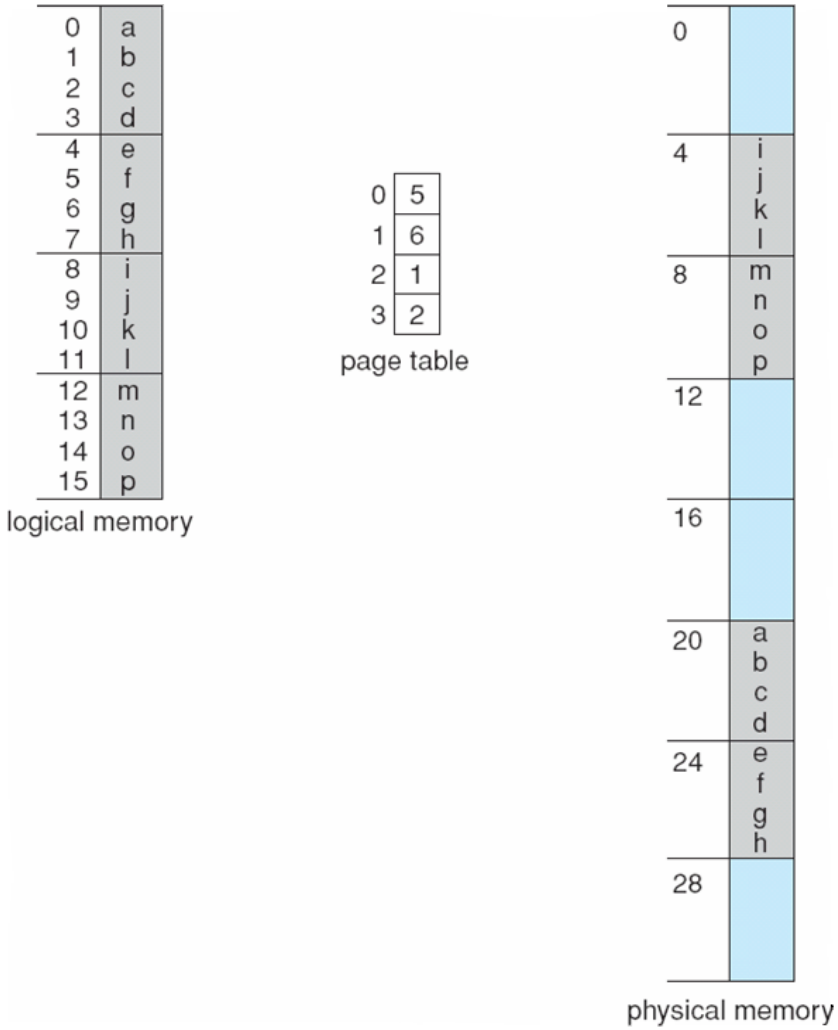


Paging Model of Logical and Physical Memory



Paging Example

- A page size of 4 bytes
- A physical memory of 32 bytes (8 frames)



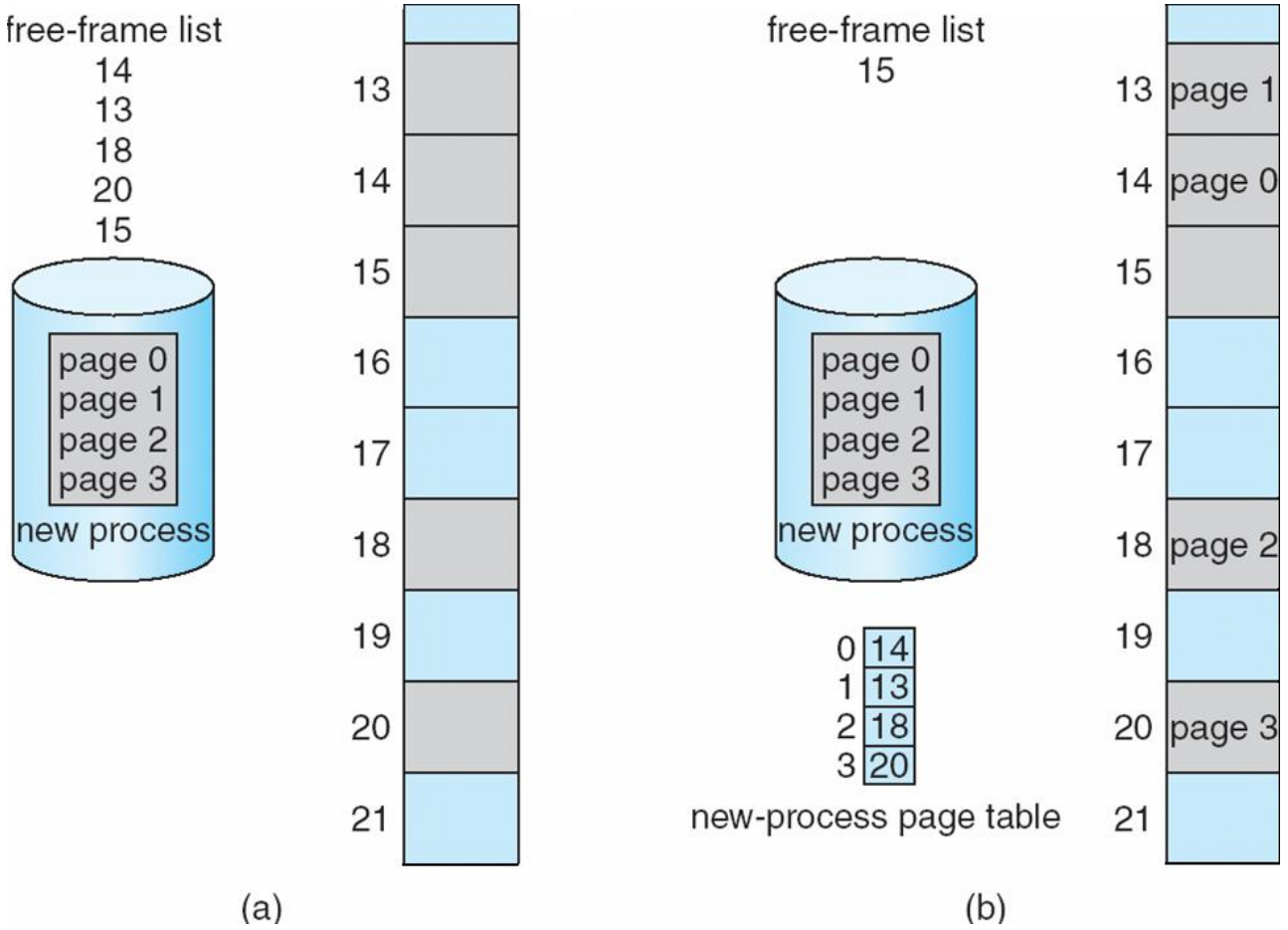


Paging – Internal Fragmentation

- **Example Calculation**
  - Page size = **2,048 bytes**
  - Process size = **72,766 bytes**
  - Needs **35 pages + 1,086 bytes**
  - Last page partially filled → **Internal fragmentation = 2,048 – 1,086 = 962 bytes**
- **General Observations**
  - **Worst case**: Almost a full frame wasted (**1 frame – 1 byte**)
  - **Average waste**: About **½ frame per process**
  - Smaller frames → **less fragmentation**, but:
    - More **page table entries** required
    - Higher **memory overhead** for tracking
- **Trend**
  - Page sizes have been increasing to balance **fragmentation vs. page table size**
  - Example: **Solaris** supports **8 KB** and **4 MB** page sizes

Allocating Free Frames

- Keep a list of free frames
- Example of frame allocation



- New process arrives: (a) before allocation (b) after allocation

Implementation of Page Table

- **Location**
  - Page table stored in **main memory**
  - **PTBR** (Page-Table Base Register) → points to page table
  - **PTLR** (Page-Table Length Register) → indicates page table size
- **Memory Access**
  - Each memory reference requires **2 accesses**:
    - One for the page table
    - One for the actual data/instruction
  - → CPU runs at **~50% effective speed**
- **Solution: TLB (Translation Lookaside Buffer)**
  - Special **fast-lookup hardware cache**
  - Stores recent page table entries
  - Reduces two-access penalty → typically **1 memory access**
  - Also called **associative memory**

Associative Memory (TLB Hardware)

What is Associative Memory?

- **Associative Memory** performs **parallel search** on stored entries
- Used in the **Translation Lookaside Buffer (TLB)**
- Searches **all entries simultaneously** instead of sequentially

Structure of Associative Memory

Page Number	Frame Number
p_0	f_0
p_1	f_1
p_2	f_2
...	...

- Stores **recent page → frame mappings**

Key Advantage

- **Very fast address translation**
- Reduces **memory access time**

Address Translation

Logical address = **(p, d)**

- **p** → page number
- **d** → page offset

Step 1: Search TLB

- If **page number p is found** → obtain **frame number f**
- Physical address = **(f, d)**

Step 2: TLB Miss

- If **p not found in TLB**
- Access **page table in main memory**
- Load mapping into **TLB**

⚡ Translation Lookaside Buffer (TLB)

A **TLB** is a small, fast cache that stores **recently used page-table entries**.

- Implemented using **associative memory**
- Stores **(page number → frame number)** mappings
- Reduces the number of memory accesses

Address Translation Using TLB

- CPU generates a **logical address (page number, offset)**
- Page number is searched in the **TLB**
- **TLB Hit**
  - Frame number found in TLB
  - Access memory directly
- **TLB Miss**
  - Page table in main memory is accessed
  - Entry is loaded into the TLB

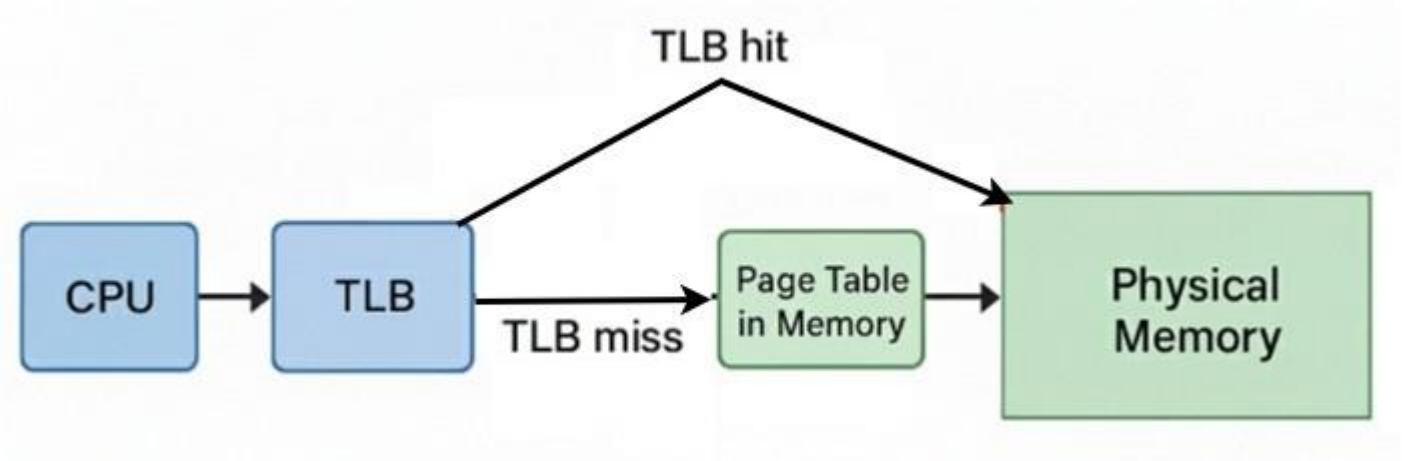
Advantages of TLB

- Reduces **effective memory access time**
- Speeds up **virtual memory address translation**
- Improves **system performance**

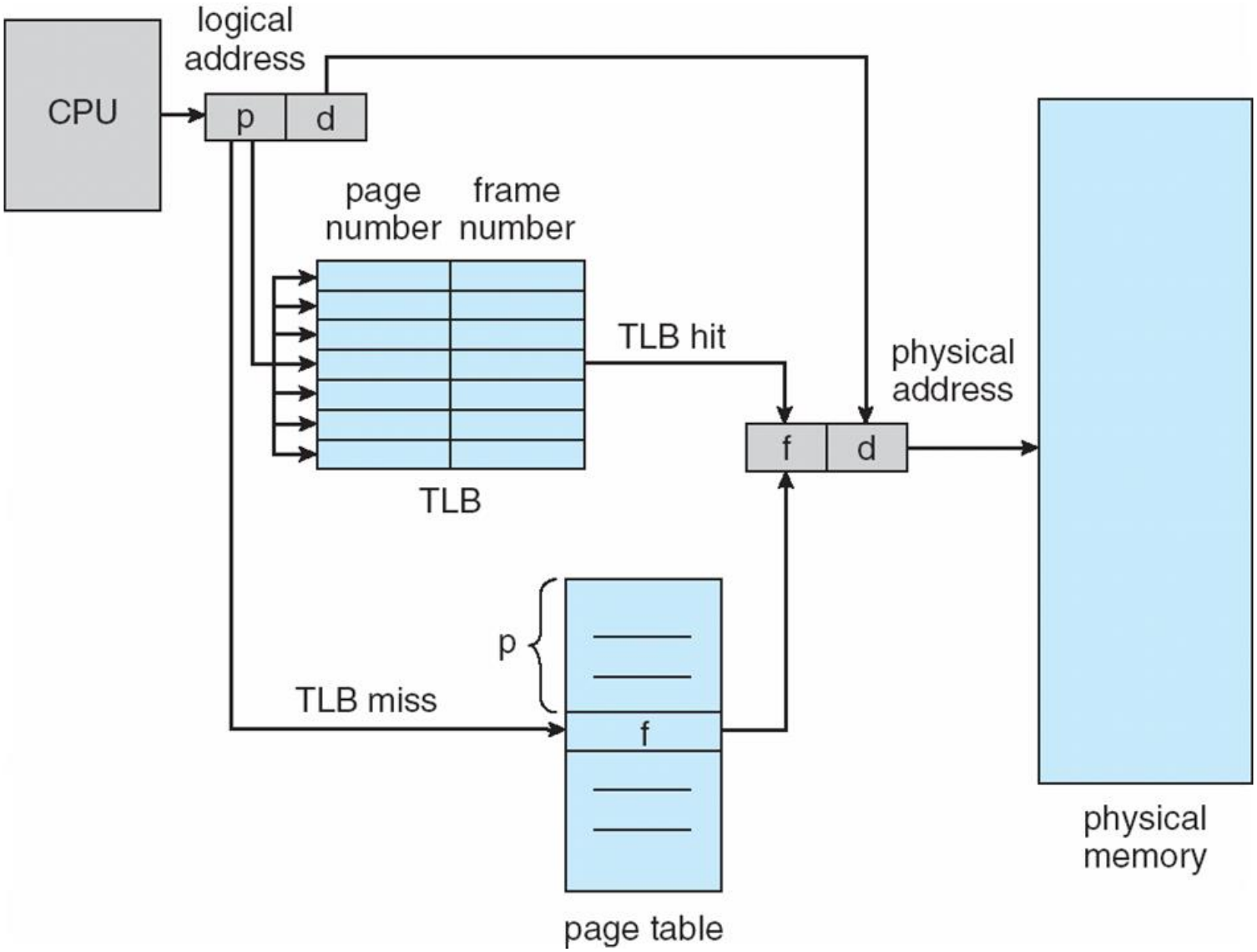
Key Idea: TLB acts as a **cache for page table entries** to speed up paging.

⚡ Translation Lookaside Buffer (TLB)

- **Size**: Typically, **64 – 1,024 entries**
- **Operation**:
  - On a **TLB hit** → fast translation
  - On a **TLB miss** → entry loaded from page table for future use
- **Management**:
  - **Replacement policy** needed (e.g., LRU, FIFO)
  - Some entries can be **wired down** (permanently reserved)
- **Address-Space Identifiers (ASIDs)**
  - Each entry may store an **ASID** → uniquely identifies a process
  - Provides **address-space protection**
  - Without ASIDs → must **flush TLB** on every context switch (costly)



Paging Hardware With TLB



Effective Access Time

TLB Hit Ratio

- **Hit Ratio (h):** Fraction of memory references found in the **TLB**
- Example: **h = 0.8** → 80% of address translations are found in the TLB

Memory Access Scenarios

1. TLB Hit (probability = h)

- Frame number found directly in **TLB**
- Only **1 memory access** required
- Access time = $\mathbf{t_m}$

2. TLB Miss (probability = 1 − h)

- Access **page table in memory**
- Then access **actual memory**
- Access time = $\mathbf{2t_m}$

Effective Access Time Formula

$$EAT = h\,(t_{TLB} + t_M) + (1-h)(t_{TLB} + 2t_M)$$

(When TLB lookup is very small and often ignored)

$$\boldsymbol{EAT = h \times t_M + (1 - h) \times 2t_M}$$

Where:

- **h** = TLB hit ratio, $\mathbf{t_m}$ = main memory access time, $\mathbf{t_{tlb}}$ → TLB lookup time

Effective Access Time

Example

$$\boldsymbol{EAT = h \times t_M + (1 - h) \times 2t_M}$$

- Memory access time = **10 ns**
- **Hit ratio = 80%**

$$EAT = 0.8 \times 10 + 0.2 \times 20 = 12\,ns$$

- **Hit ratio = 99%**

$$EAT = 0.99 \times 10 + 0.01 \times 20 = 10.1\,ns$$

Key Insight

- **Higher TLB hit ratio → Faster memory access**
- TLB greatly **reduces average memory access time**

Example

Given

- **Memory access time (t_m)** = 100 ns
- **TLB lookup time ($t_{tlβ}$)** = 20 ns
- **TLB hit ratio (h)** = 0.9

Effective Access Time Formula

$$EAT = h \times t_M + (1 - h) \times 2t_M$$

Step 1: Substitute Values

- EAT = 0.9 (20 + 100) + 0.1 (20 + 200)
- EAT = 0.9 × 120 + 0.1 × 220
- EAT = 108 + 22
- **EAT = 130 ns**

Interpretation:

Case	Access Time
TLB Hit	$t_{TLB} + t_M$
TLB Miss	$t_{TLB} + 2t_M$

Conclusion

- Without TLB → **200 ns** (two memory accesses)
- With TLB → **130 ns**

✅ **TLB significantly reduces memory access time.**

Example-1

- Consider a two-level paging hardware with a TLB. Assume that the entire page table and all the pages are in the main memory. It takes 70 milliseconds to access the main memory, and the effective memory access time is measured to be 118 milliseconds. If the TLB hit ratio is 90%, then what is the TLB access time (in milliseconds)?

Given

- Main memory access time $t_M = 70$ms, TLB hit ratio $h = 0.9$,
- Effective Access Time $EAT = 118$ms
- For **two-level paging with TLB**:
- **TLB Hit:**
  TLB lookup + **1 memory access**
  Time = $t_{TLB} + t_M$
- **TLB Miss:**
  TLB lookup + **two page table accesses** + **data access**
  Time = $t_{TLB} + 3t_M$

Effective Access Time Formula

$$EAT = h(t_{TLB} + t_M) + (1 - h)(t_{TLB} + 3t_M)$$

Substitute the values

$$118 = 0.9(t_{TLB} + 70) + 0.1(t_{TLB} + 210)$$

- Expand:
- $118 = 0.9t_{TLB} + 63 + 0.1t_{TLB} + 21$

Memory Protection

Purpose

- Ensures a process **accesses only its own memory**
- Prevents **illegal access** to other processes or OS memory

Protection Mechanism

- Each page/frame has **protection bits**
- These bits control **allowed operations**

Examples: Read-only, Read–write, Execute-only

Valid–Invalid Bit (Page Table)

- Each page table entry has a **valid/invalid bit**

Valid (1):

- Page is part of the process's **logical address space**
- Access is **allowed**

Invalid (0):

- Page is **not part of the process**
- Access is **not allowed**

Key Idea

Memory protection ensures **safe and secure execution** of processes

Address Space Control

- **Page-Table Length Register (PTLR)** → Ensures process accesses only valid pages

Protection Violation

- Illegal access triggers a **trap (exception)**
- Control transferred to the **Operating System**

Valid (v) or Invalid (i) Bit In A Page Table

How It Works

1. CPU generates logical address **(page number p, offset d)**
2. Page table is checked:
   - If **Valid bit = 1** → Access allowed
   - If **Valid bit = 0** → **Trap to OS**

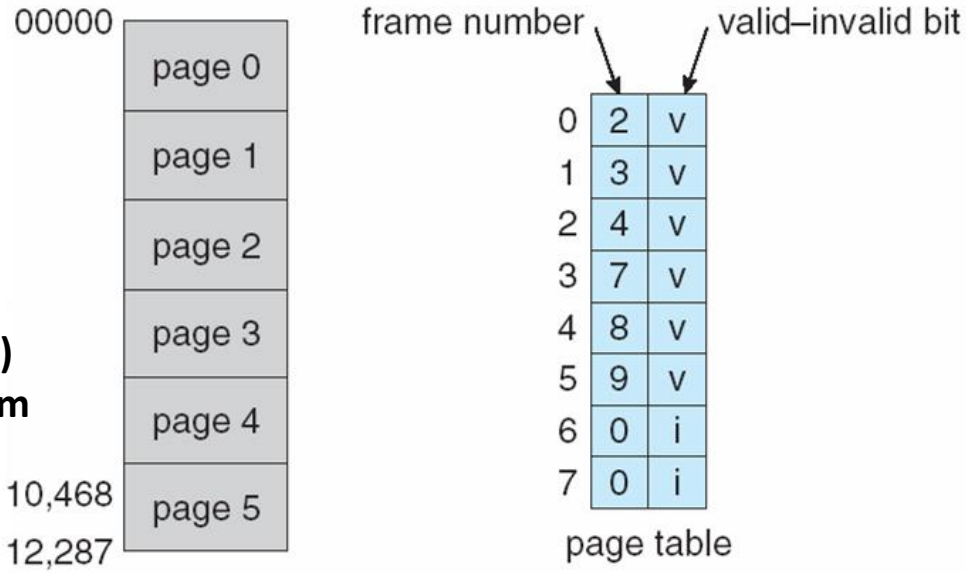
Protection Flow

•Valid Page

→ Frame number obtained
→ Physical address formed
→ Memory access continues

•Invalid Page

→ Hardware raises **trap (exception)**
→ Control goes to **Operating System**



Key Components

- **Page Table Entry (PTE) includes:**
  - Frame number
  - **Valid/Invalid bit**
  - Protection bits (R/W/X)

Shared Pages

Shared Code (Reentrant Code)

- **Single copy** of read-only code shared among multiple processes
- Code must be **reentrant** (no self-modifying data)
- Saves **memory space**

Examples:

- Text editors, Compilers, Window systems

Why Shared Code?

- Reduces **memory usage**
- Improves **efficiency**
- Similar to **threads sharing code in a process**

Shared Data (Optional)

- Some systems allow **read–write shared pages**
- Enables **inter-process communication (IPC)**

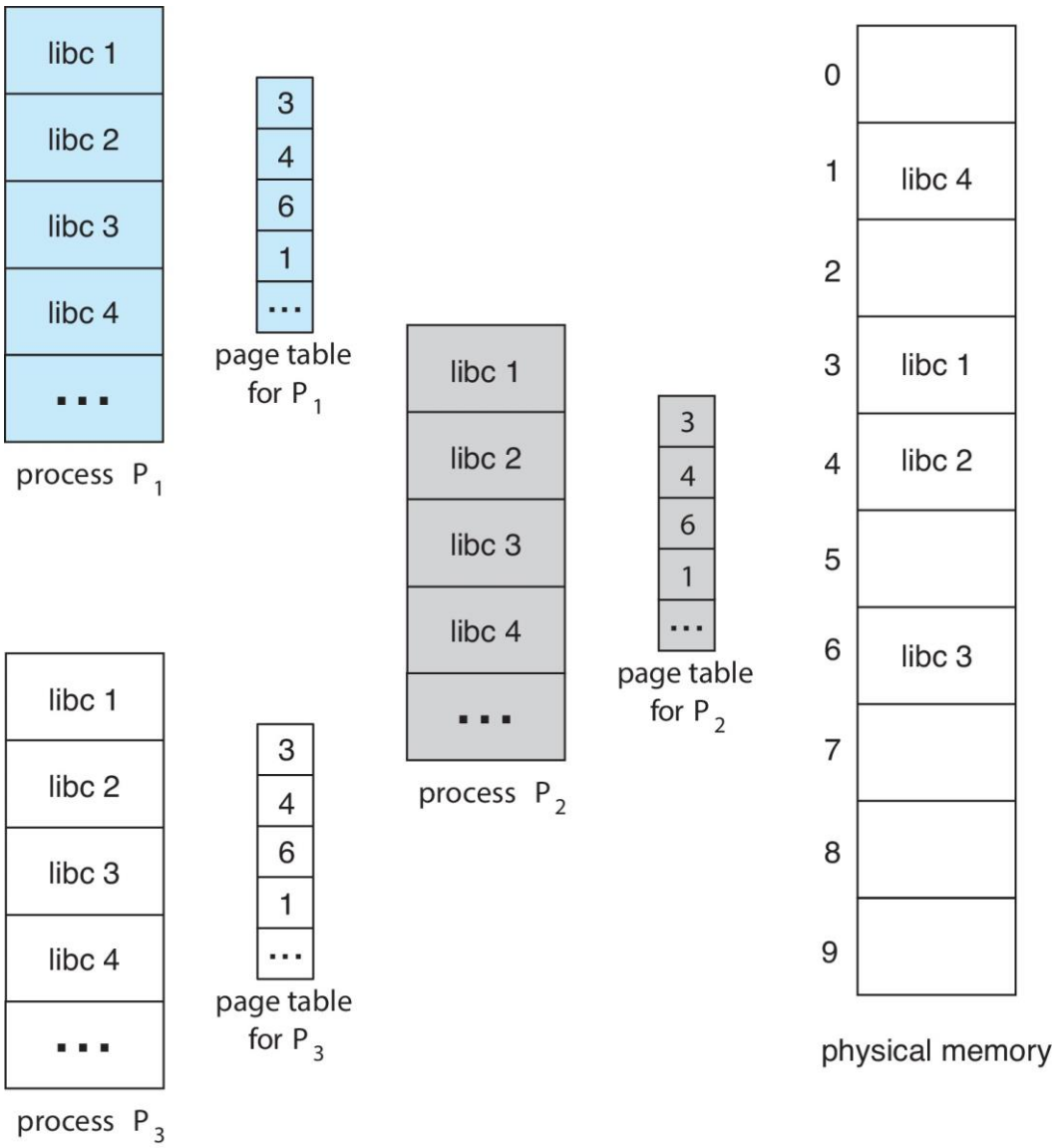
Private Code & Data

- Each process has its **own copy**
- Includes:
  - Stack, Heap, Private variables
- Can be placed **anywhere in logical address space**

✅ **Key Idea**

Code can be shared → Data is usually private

Shared Pages Example

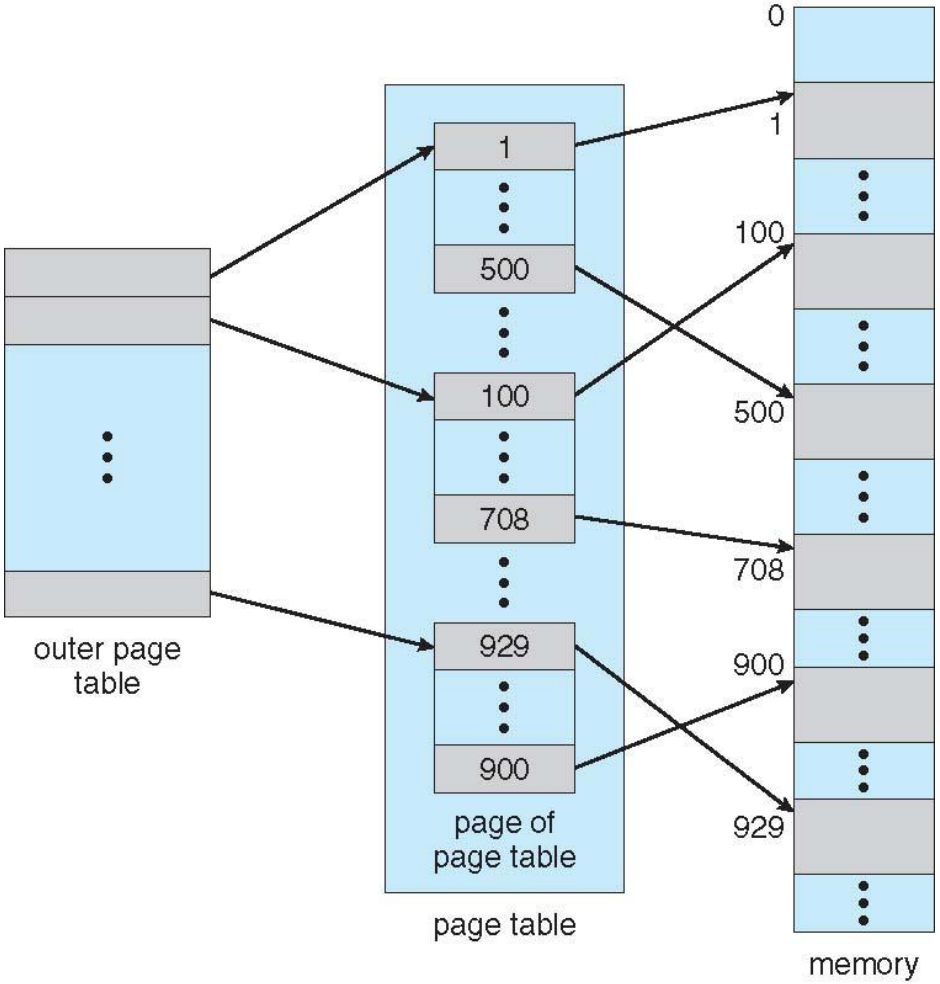


Structure of the Page Table

- Memory structures for paging can get huge using straight-forward methods
  - Consider a 32-bit logical address space as on modern computers
  - Page size of 1 KB (2^{10})
  - Page table would have 1 million entries (2^{32} / 2^{10})
  - If each entry is 4 bytes ➔ each process requires 16 MB of physical address space for the page table alone
    - Do not want to allocate that contiguously in main memory
  - One simple solution is to divide the page table into smaller units
    - Hierarchical Paging
    - Hashed Page Tables
    - Inverted Page Tables

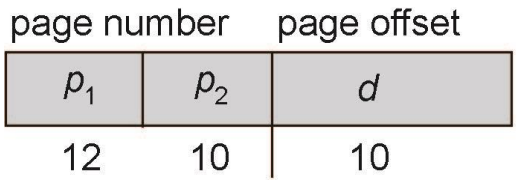
Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table



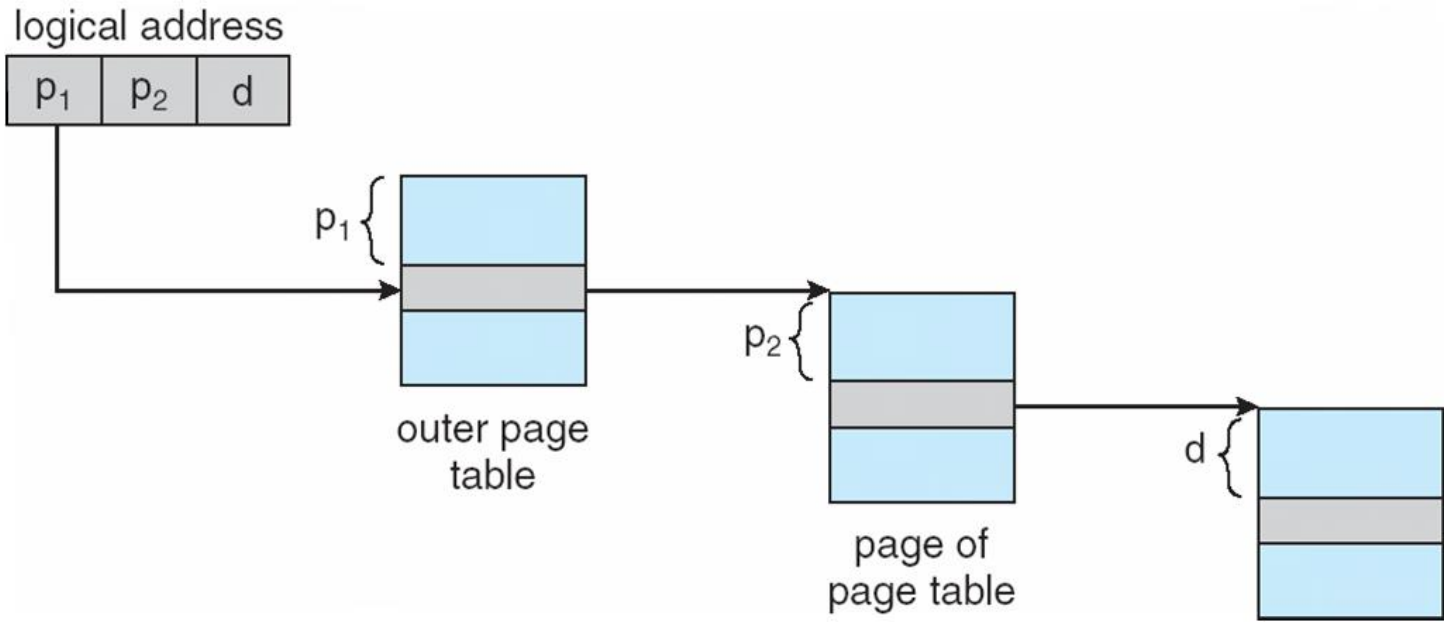
Two-Level Paging Example

- A logical address (on 32-bit machine with 1K page size) is divided into:
  - A page number consisting of 22 bits
  - A page offset consisting of 10 bits
- Since the page table is paged, the page number is further divided into:
  - A 12-bit page number
  - A 10-bit page offset
- Thus, a logical address is as follows:



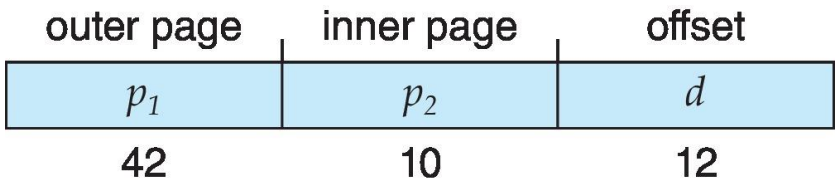
- where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table
- Known as **forward-mapped page table**

Address-Translation Scheme



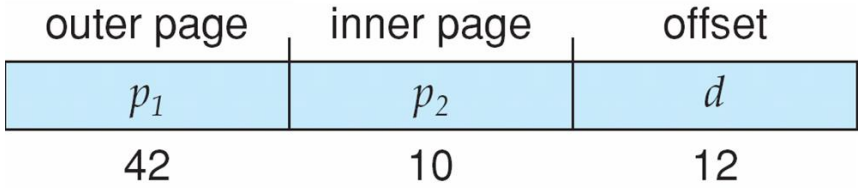
64-bit Logical Address Space

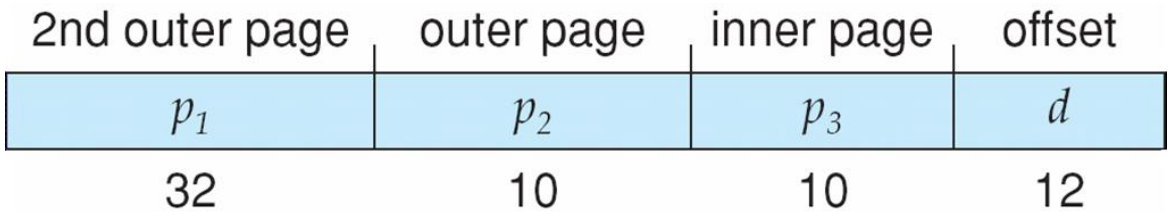
- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
  - Then page table has 2^{52} entries
  - If two level scheme, inner page tables could be 2^{10} 4-byte entries
  - Address would look like



  - Outer page table has 2^{42} entries or 2^{44} bytes
  - One solution is to add a 2nd outer page table
  - But in the following example the 2nd outer page table is still 2^{34} bytes in size
    - And possibly 4 memory access to get to one physical memory location

Three-level Paging Scheme

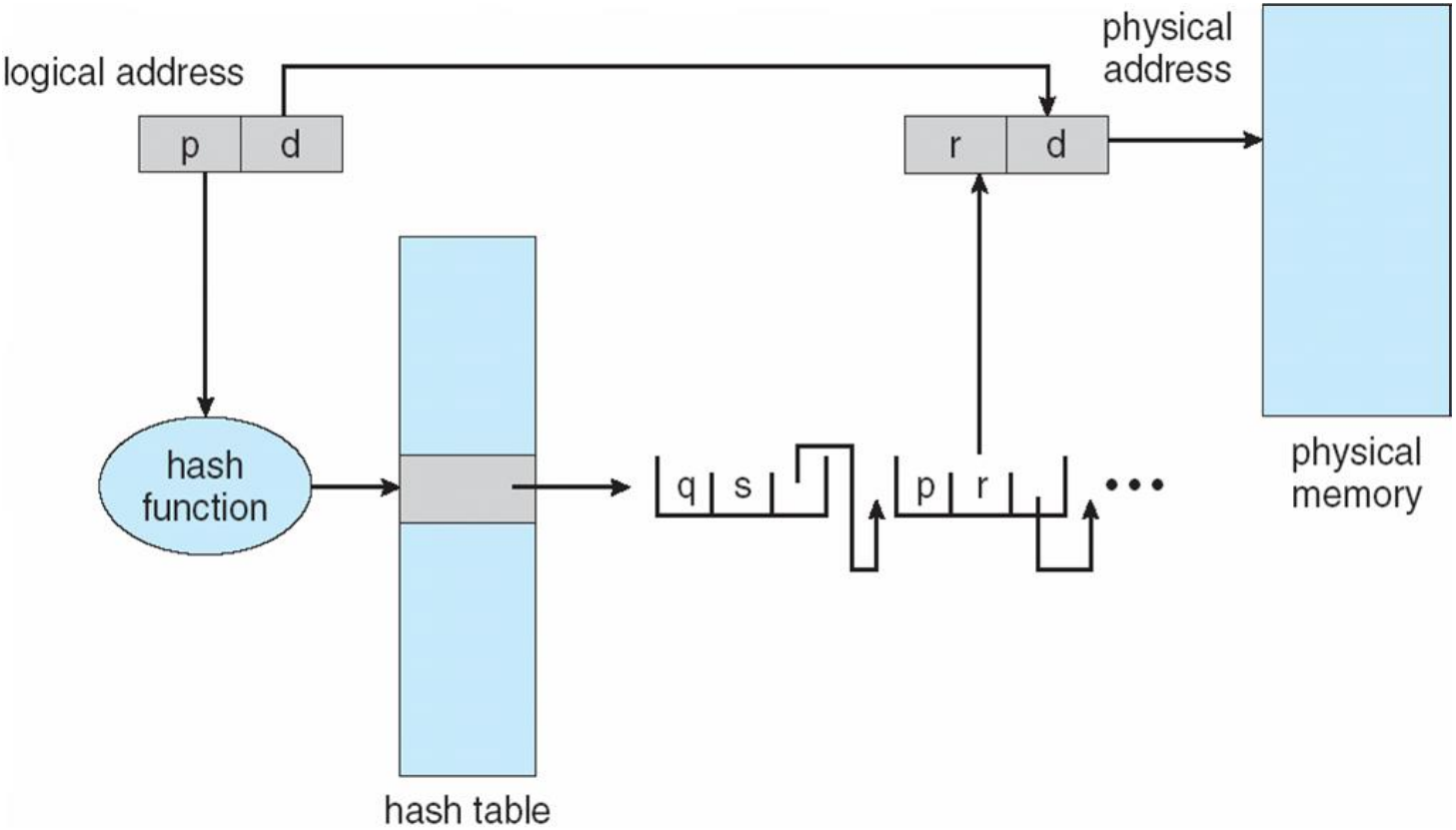




Hashed Page Tables

- Used in architecture with address spaces > 32 bits
- The virtual page number is hashed into a page table
  - This page table contains a chain of elements hashing to the same location
- Each element contains
  1. The virtual page number
  2. The value of the mapped page frame
  3. A pointer to the next element
- Virtual page numbers are compared in this chain searching for a match
  - If a match is found, the corresponding physical frame is extracted

Hashed Page Table Hardware

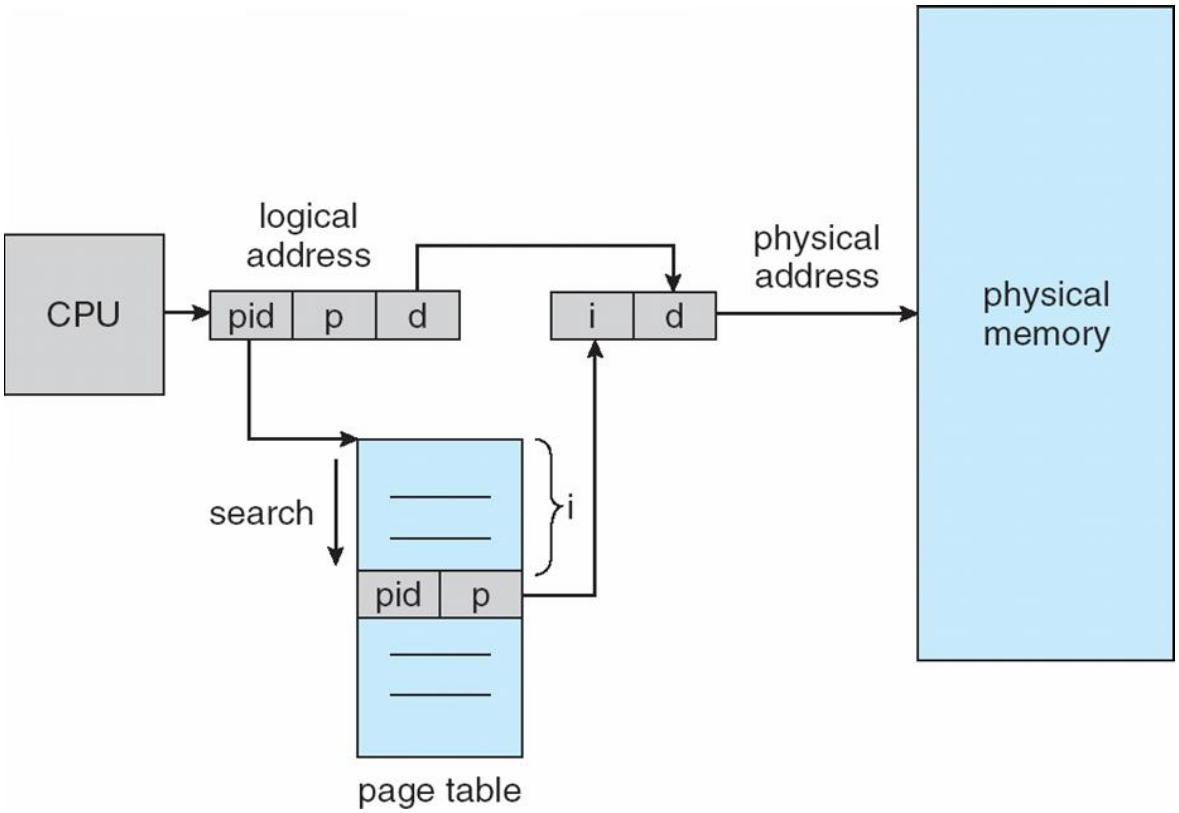


Clustered Hashed Page Tables

- Variation for 64-bit addresses
- Similar to hashed but each entry refers to several pages (such as 16) rather than 1
- Especially useful for **sparse** address spaces (where memory references are non-contiguous and scattered)

Inverted Page Table

- Rather than having each process keep a page table and track of all possible logical pages, track all physical pages
- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page



Inverted Page Table (Cont.)

- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs
- Use hash table to limit the search to one (or at most a few) page-table entries
  - TLB can accelerate access
- But how to implement shared memory?
  - One mapping of a virtual address to the shared physical address

Swapping

What is Swapping?

- A process is **temporarily moved** from main memory → **disk (backing store)**
- Later brought **back into memory** to resume execution

Backing Store

- **Fast disk** used to store process memory images
- Must support **direct access** for efficient swapping

Roll Out / Roll In

- Used in **priority-based scheduling**
- **Low-priority process → swapped out**
- **High-priority process → loaded into memory**

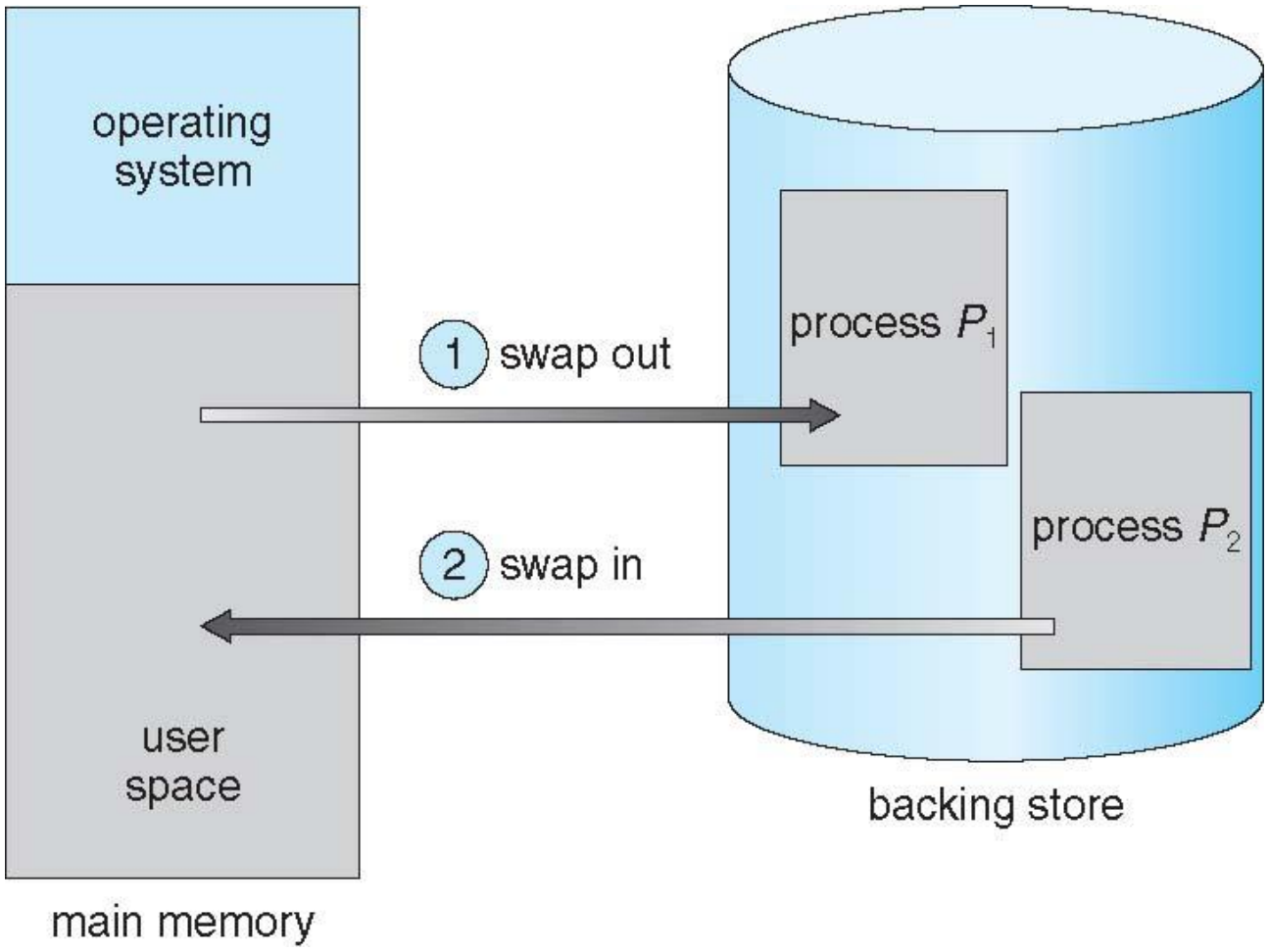
Performance Consideration

- **Swap time ≈ Disk transfer time**
- Depends on:
  - Process size
  - Disk speed

Ready Queue

- Stores **processes ready to execute**
- Some processes may be **on disk (swapped out)**

Schematic View of Swapping



Swapping (Cont.)

Address Binding & Relocation

- Does a swapped-out process return to the **same physical address?**
- ✅ **Depends on address binding:**
  - **Static binding** → Must return to same location
  - **Dynamic binding (with relocation)** → Can load anywhere

Swapping in Modern Systems

- Used in systems like **UNIX, Linux, Windows**
- Typically:
  - **Disabled by default**
  - Activated when **memory usage exceeds threshold**
  - Disabled again when memory pressure decreases

Constraints on Swapping

1. Pending I/O

- Cannot swap out a process doing I/O
- Data may go to **wrong memory location**

2. Double Buffering

- Solution: I/O via **kernel buffer**
- Flow: Process → Kernel → Device
- ❗ Adds **extra overhead**

Context Switch Time (with Swapping)

When Does Swapping Occur?

- If the next process is **not in memory**
- OS must:
  - **Swap out** current process
  - **Swap in** next process

Impact on Context Switch

- Context switch becomes **very slow**
- Dominated by **disk I/O time**, not CPU time

Example

- Process size = **100 MB**
- Disk transfer rate = **50 MB/sec**

Swap Out Time:

$$100/50 = 2\ sec = 2000\ ms$$

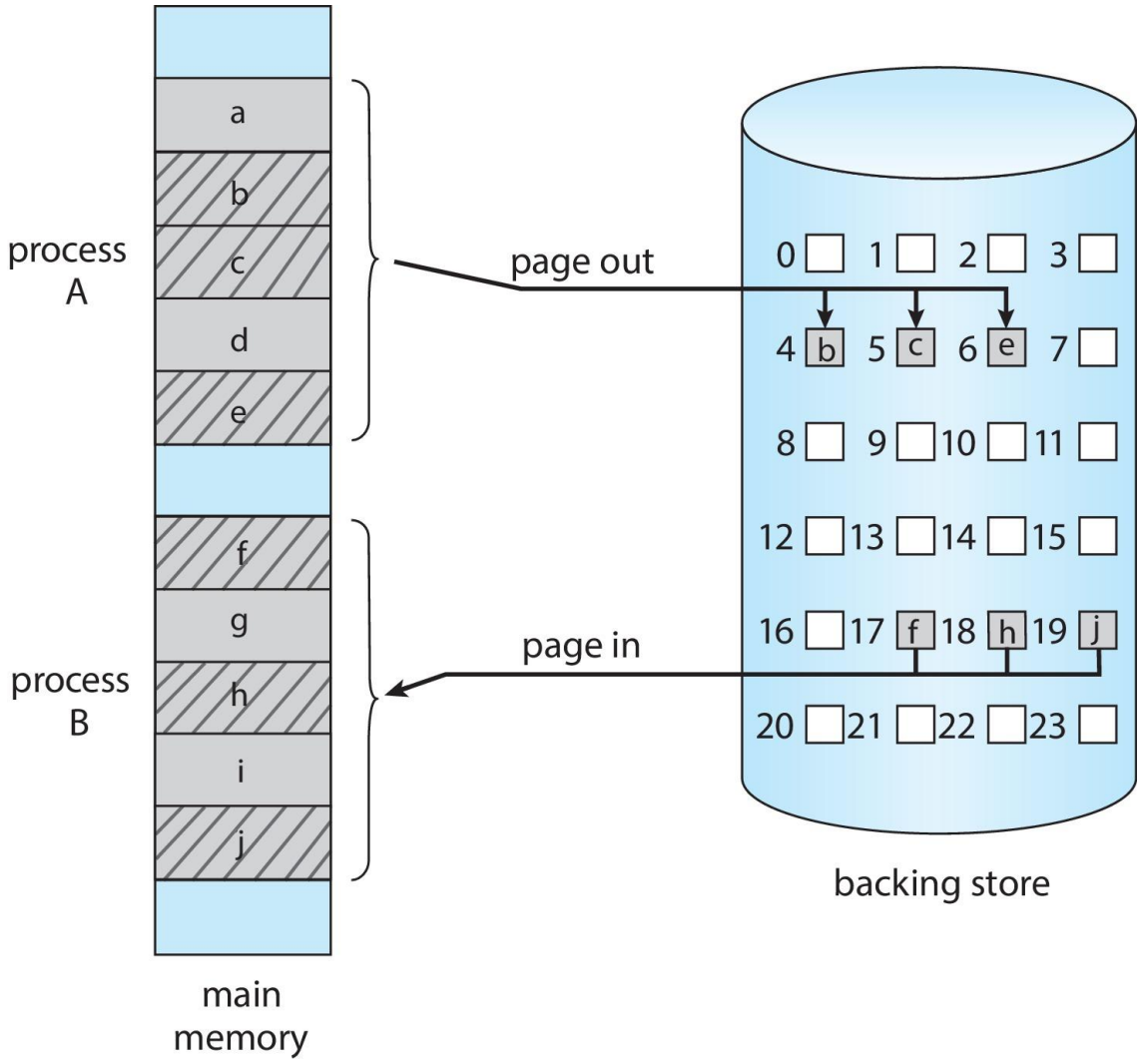
Swap In Time:

$$100/50 = 2\ sec = 2000\ ms$$

Total Context Switch Time

$$\text{Total} = 2000 + 2000 = 4000\ ms = 4\ sec$$

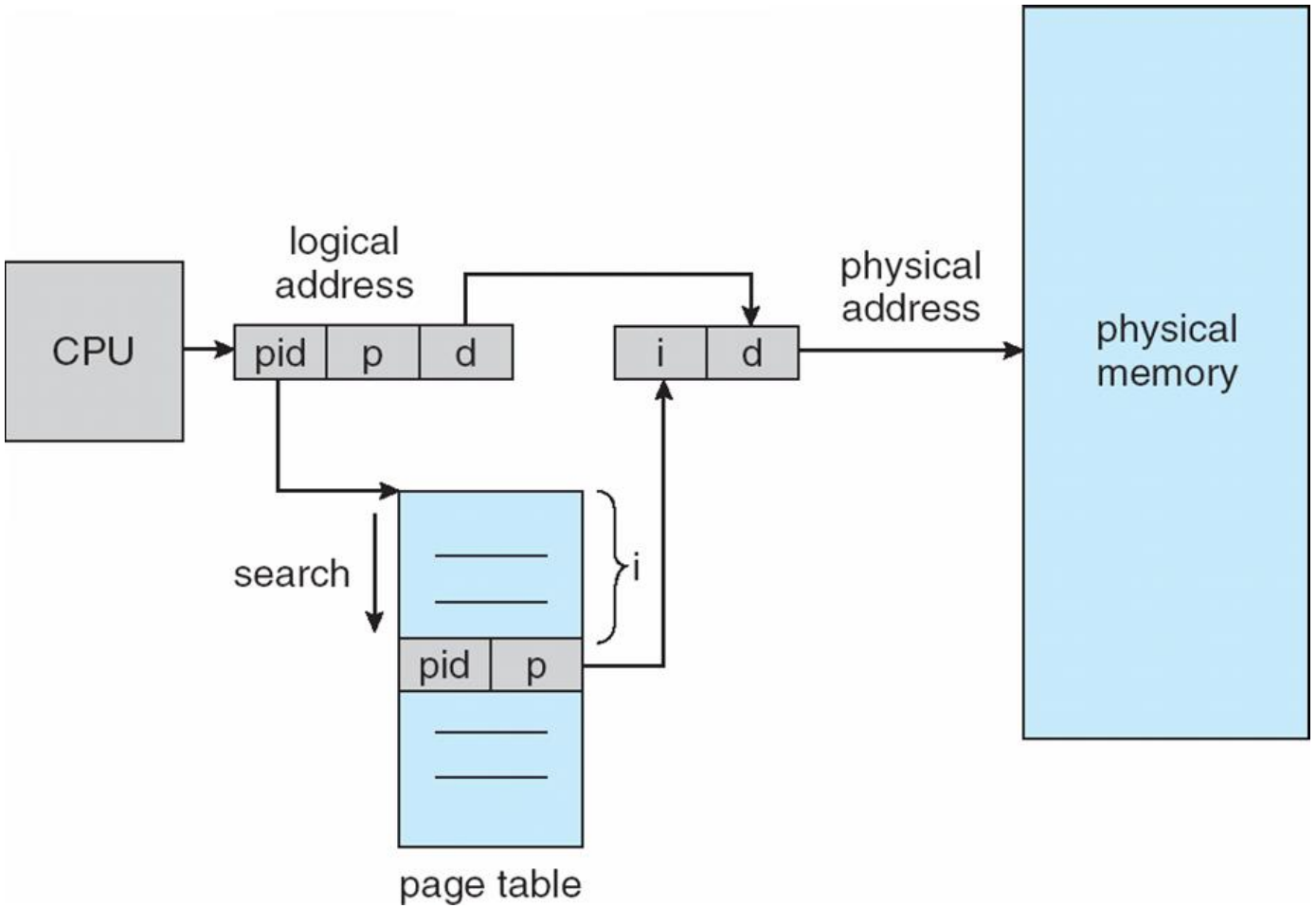
Swapping with Paging



Swapping on Mobile Systems

- Not typically supported -- Flash memory based
  - Small amount of space
  - Limited number of write cycles
  - Poor throughput between flash memory and CPU on mobile platform
- Instead use other methods to free memory if low
  - iOS ***asks*** apps to voluntarily relinquish allocated memory
    - Read-only data thrown out and reloaded from flash if needed
    - Failure to free can result in termination
  - Android terminates apps if low free memory, but first writes **application state** to flash for fast restart
  - Both OSes support paging as discussed below

Inverted Page Table Architecture



Swapping (Cont.)

- Other constraints on swapping
  - Pending I/O – can't swap out as I/O would occur to wrong process
  - Or always transfer I/O to kernel space, then to I/O device
    - Known as **double buffering**, adds overhead
- Standard swapping not used in modern operating systems
  - But modified version common
    - Swap only when free memory extremely low

Thanks